Wipro Training Program

Java - J2EE Engg

PHASE-I

Java Fundamentals
OOPS / Inheritance
Eclipse Overview
Abstraction /Packages / Exception Handling
Collection Framework
Wrapper Classes, I/O Streams, Annotation
ASSESSMENT MILESTONE - 1
Coding Standards / Best Practices
Junit
Multithreading I / II
RDBMS / SQL / PL/SQL
JDBC
ASSESSMENT MILESTONE - 2
ANT
HTML
JavaScript / CSS
Servlets and JSP
XML-I and XML-II
AJAX
ASSESSMENT MILESTONE - 3

PHASE-II

Hibernate
Spring Core
Spring Boot
REST Web Service
Angular
ASSESSMENT MILESTONE - 4

JAVA FUNDAMENTALS

JAVA

- Java is a high-level programming language originally developed by Sun Microsystems and released in 1995.
- James Gosling initiated Java language project in June 1991.
- Java runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.
- The language, initially called ‘Oak’ after renamed as Java
- Sun released the first public implementation as Java 1.0 in 1995.

JAVA TECHNOLOGIES

- JAVA SE(STARDARD EDITION)-FORMERLY J2SE
- JAVA EE(ENTERPRISE EDITION)-FORMERLY J2EE
- JAVA ME(MOBILE EDITION)-FORMERLY J2ME(MICRO EDITION)

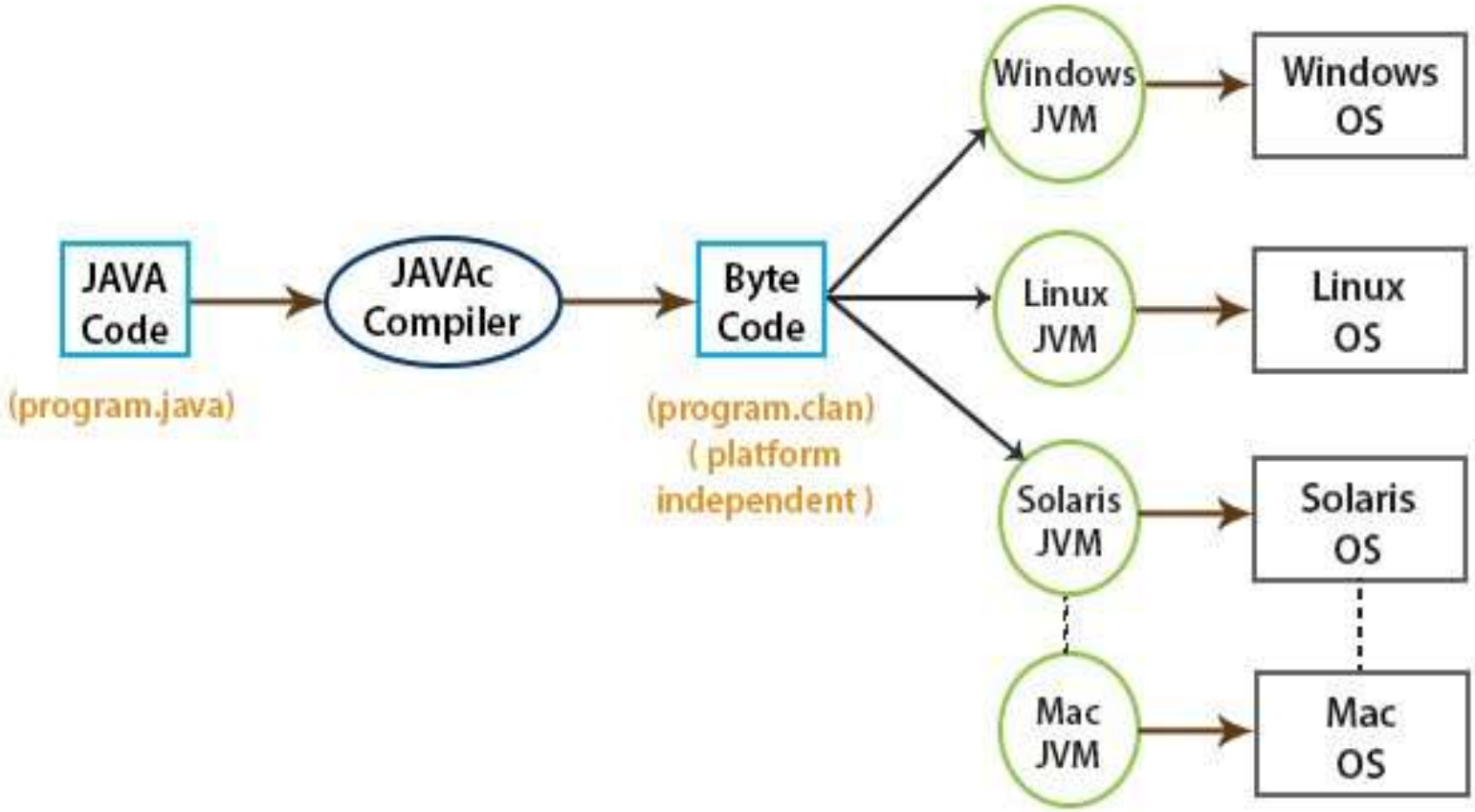
Java History

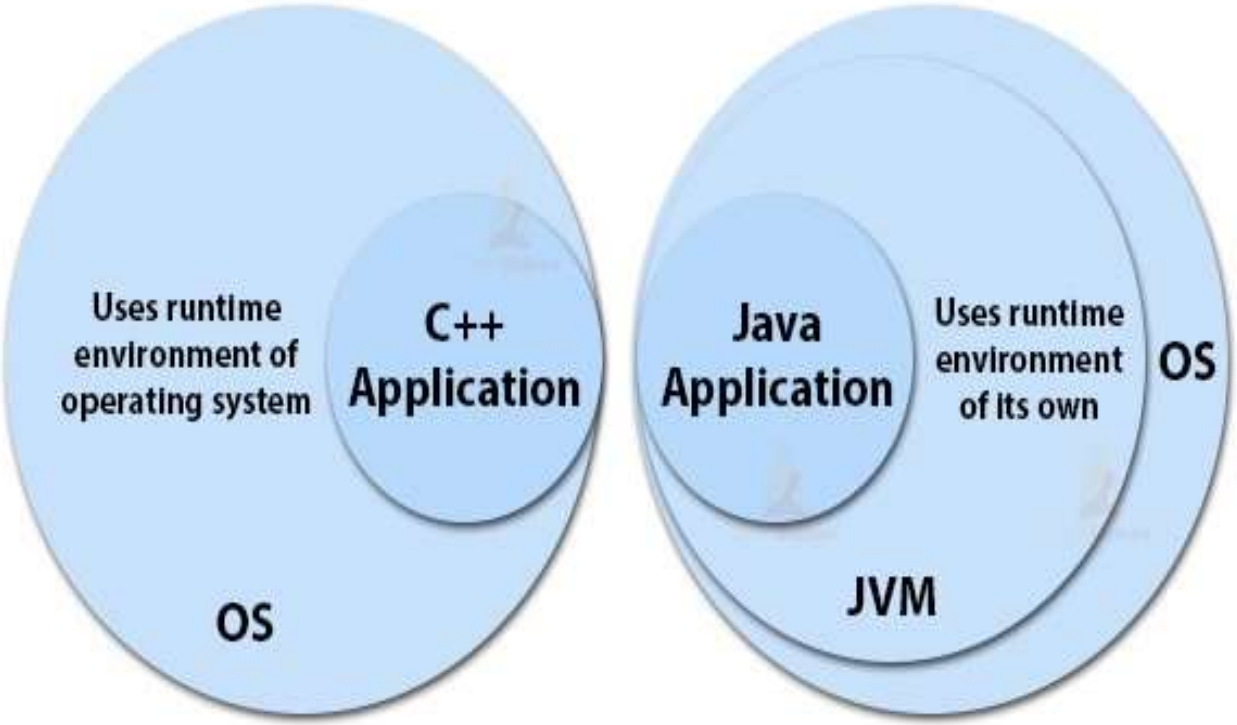
Short Java IP Story Time-line	
1996	Java released under proprietary license
1998	JCP announced, formalized
1998	First alternative Java implementation (GNU Classpath)
2001	Sun wins 1997-filed trademark lawsuit against Microsoft
2005	Apache Harmony project started
2007	Sun finishes releasing Java under GPLv2, OpenJDK
2007	Java TCK includes field-of-use restrictions
2010	Oracle acquires Sun
2010	IBM leaves Apache Harmony for OpenJDK
2010	Apache Software Foundation protests JCP

Features of Java

- ✓ Object Oriented
- ✓ Platform Independent
- ✓ Simple
- ✓ Secure
- ✓ Portable
- ✓ Robust
- ✓ Multithreaded
- ✓ Architecture-neutral
- ✓ Interpreted
- ✓ High Performance
- ✓ Distributed
- ✓ Dynamic

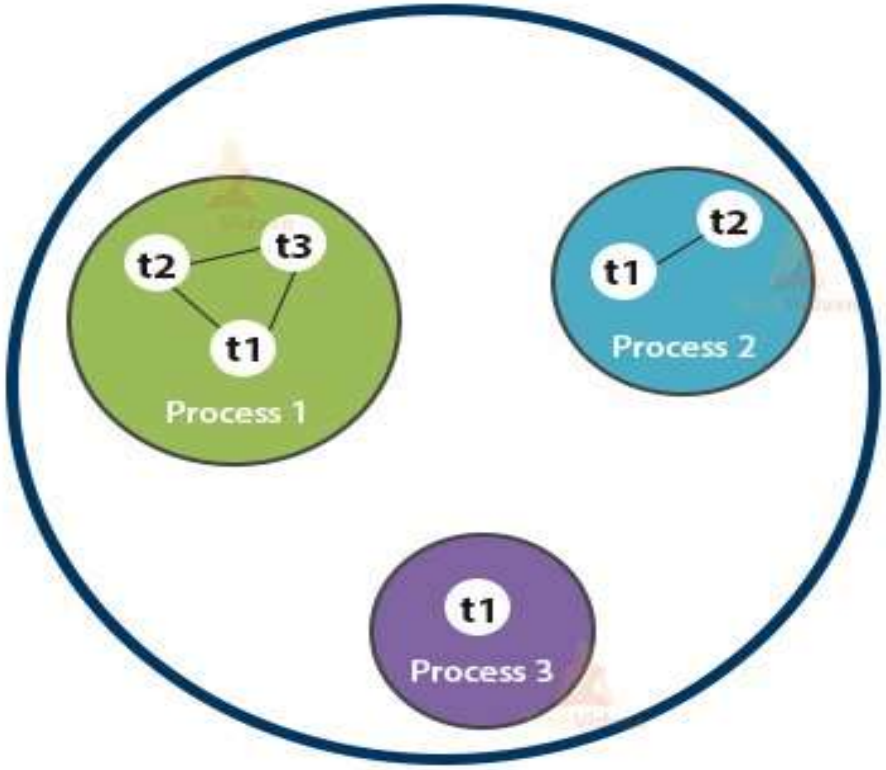
Platform Independence in Java





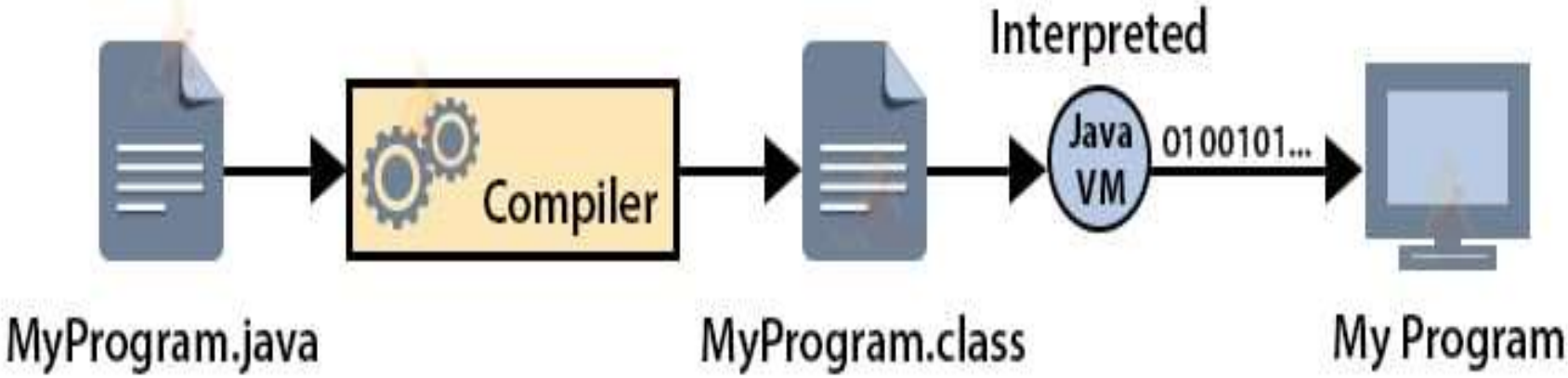
Security in Applications

Multithreading in Java



Operating System

Working of Java Virtual Machine



Programming Scenario

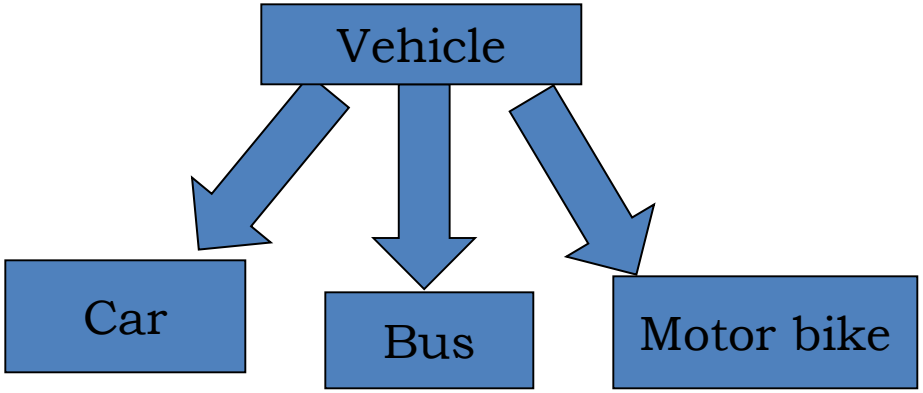
- ✓ **Structured Programming Approach**
- ✓ **Object Oriented Programming Approach**

Object oriented

What is Object Orientation ??

Similar kinds of Objects which have common properties and behavior can be group together as a single unit in a class. This approach is known as Object Orientated Approach.

Example:-



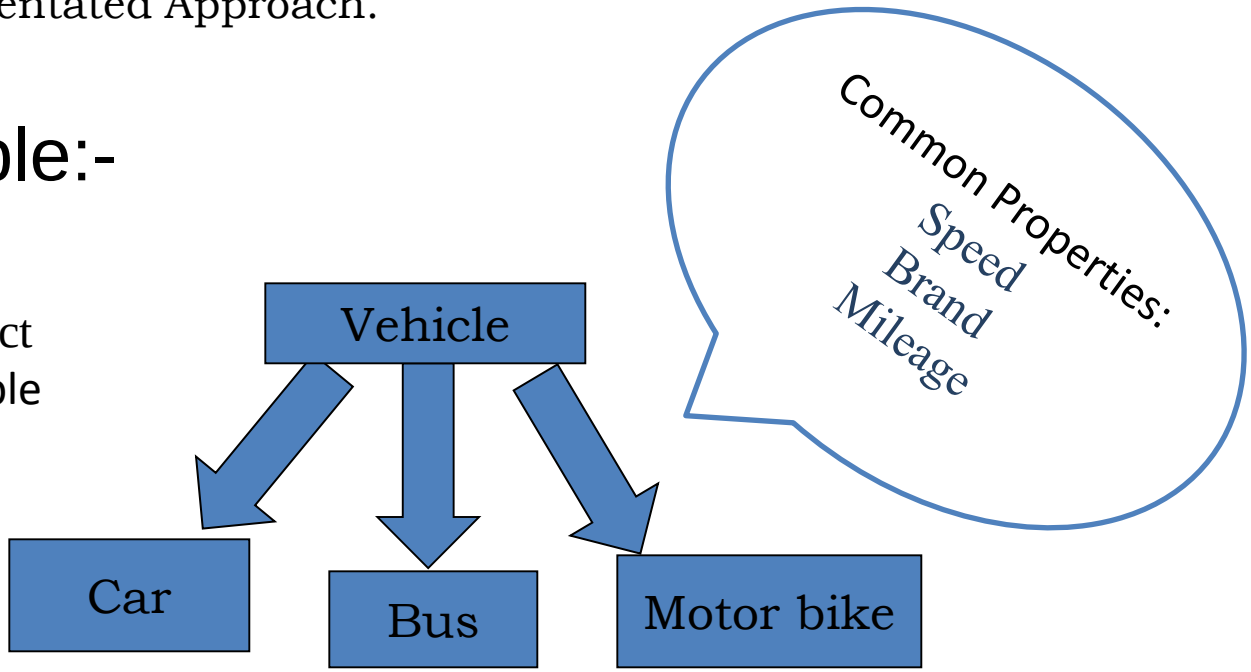
Object oriented

What is Object Orientation ??

Similar kinds of Objects which have common properties and behavior can be group together as a single unit in a class. This approach is known as Object Orientated Approach.

Example:-

Real Life Object
State - Variable



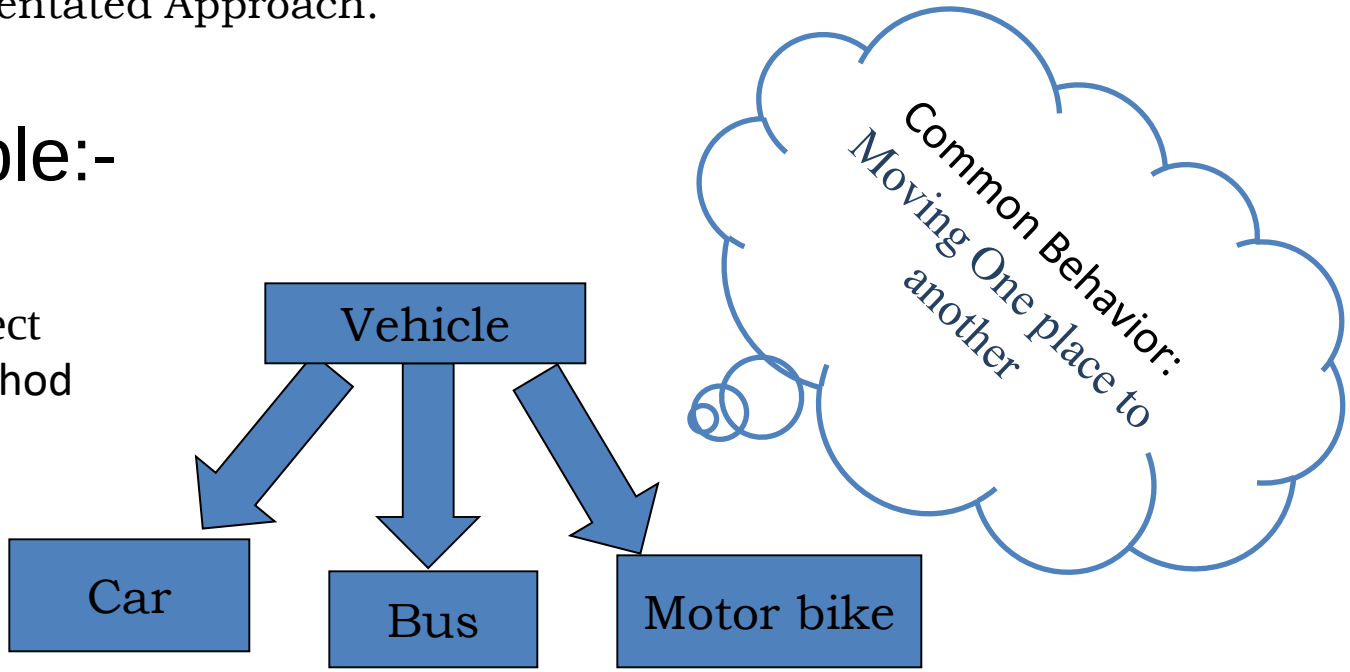
Object oriented

What is Object Orientation ??

Similar kinds of Objects which have common properties and behavior can be group together as a single unit in a class. This approach is known as Object Orientated Approach.

Example:-

Real Life Object
Behavior - method

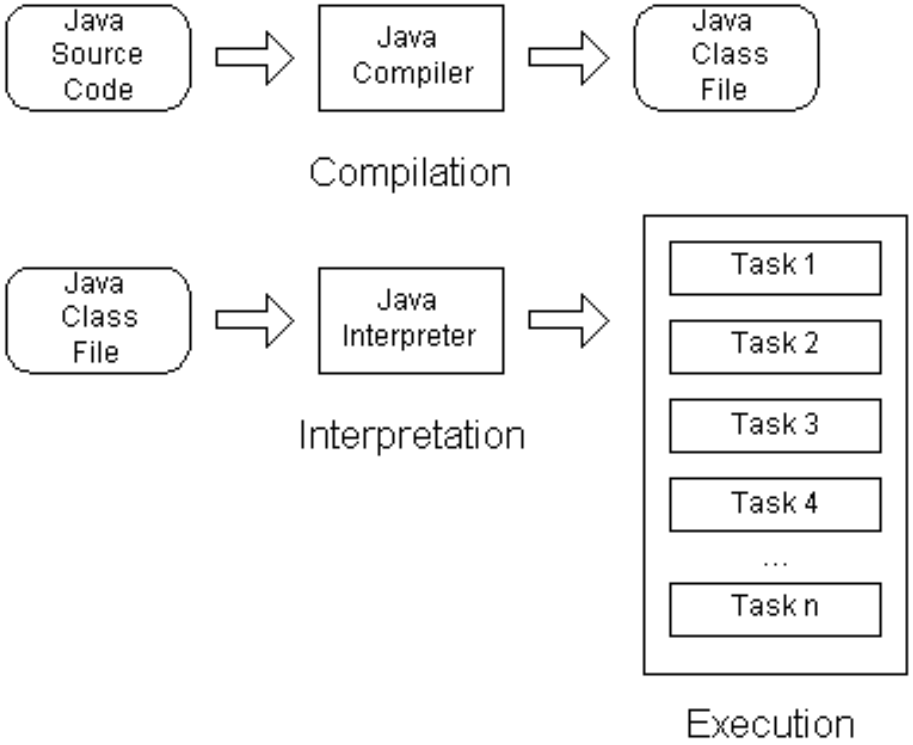


- **Popular Java Editors**
  - **Netbeans**
  - **Notepad++**
  - **Eclipse**

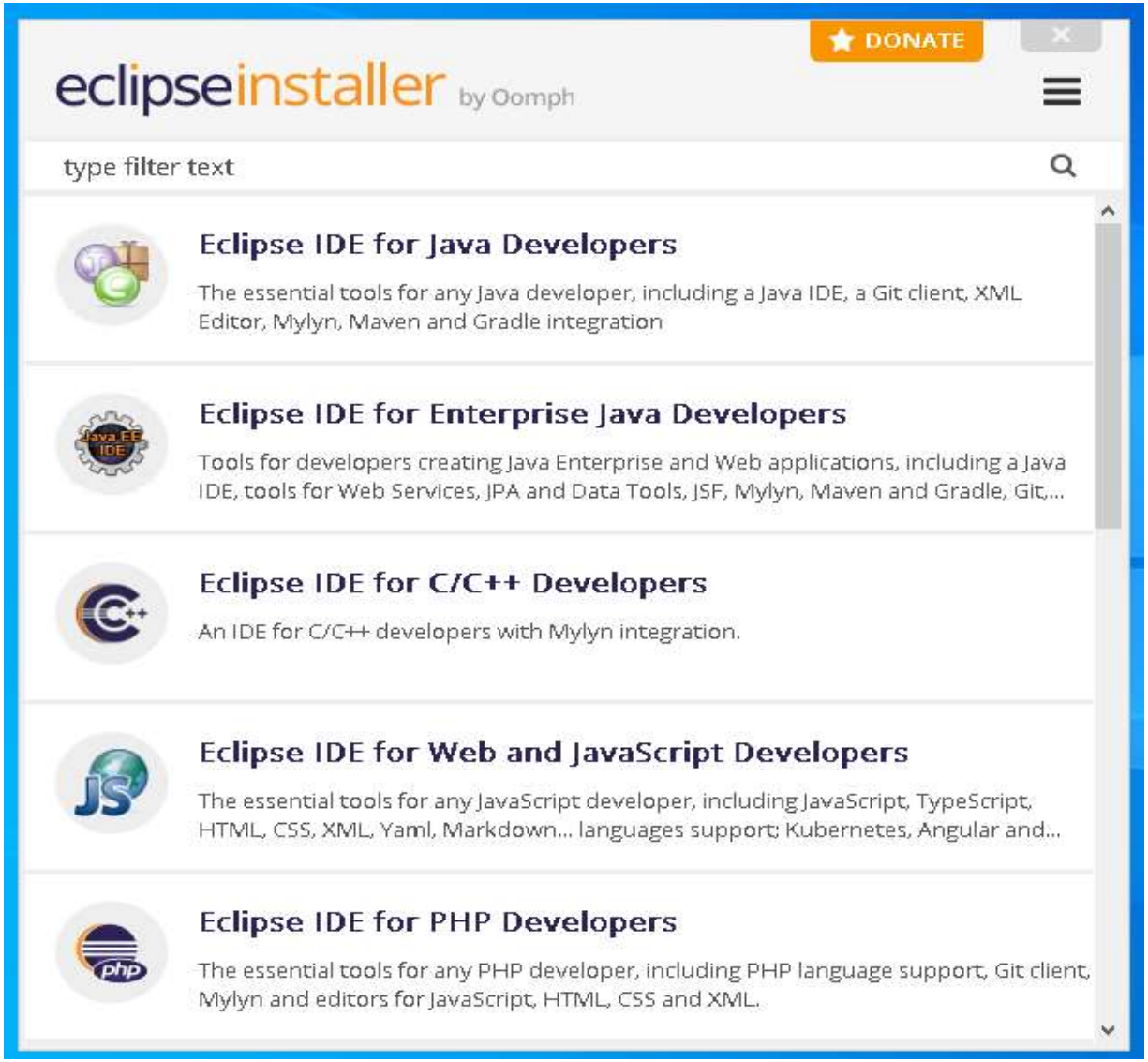
How I'm working

Step by step java program execution

JAVA



ECLIPSE IDE


DONATE
eclipseinstaller by Oomph
type filter text
Eclipse IDE for Java Developers
The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Mylyn, Maven and Gradle integration
Eclipse IDE for Enterprise Java Developers
Tools for developers creating Java Enterprise and Web applications, including a Java IDE, tools for Web Services, JPA and Data Tools, JSF, Mylyn, Maven and Gradle, Git,...
Eclipse IDE for C/C++ Developers
An IDE for C/C++ developers with Mylyn integration.
Eclipse IDE for Web and JavaScript Developers
The essential tools for any JavaScript developer, including JavaScript, TypeScript, HTML, CSS, XML, Yaml, Markdown... languages support; Kubernetes, Angular and...
Eclipse IDE for PHP Developers
The essential tools for any PHP developer, including PHP language support, Git client, Mylyn and editors for JavaScript, HTML, CSS and XML.

Environment Setup

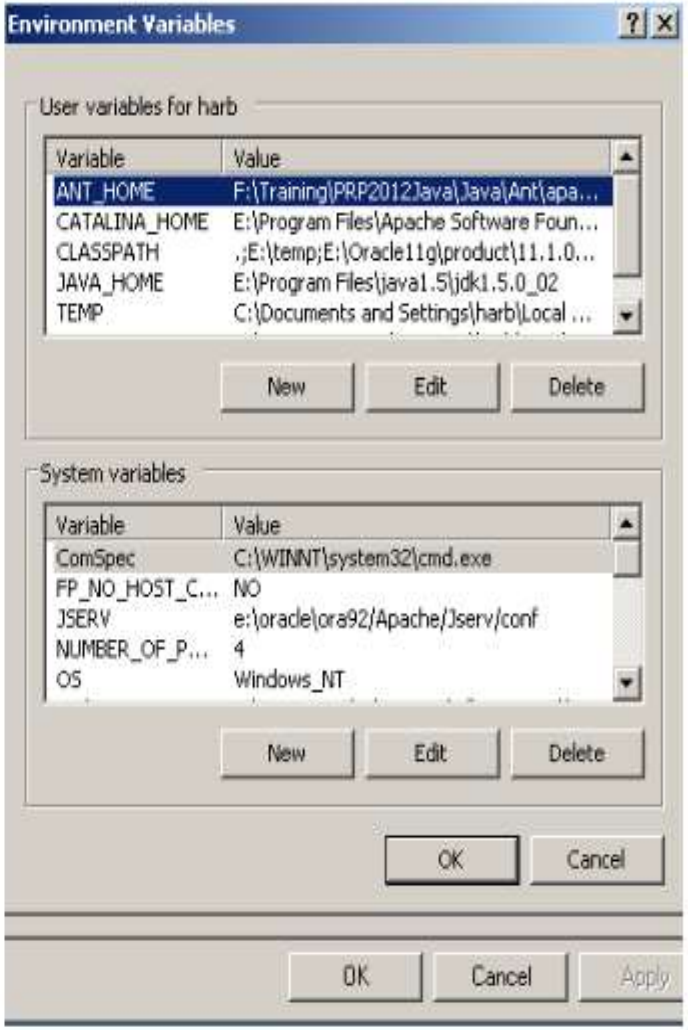
A Simple Java Program – The Set Up

- Before we learn about writing a simple java program, let us understand what preparations are needed for running a java program from console

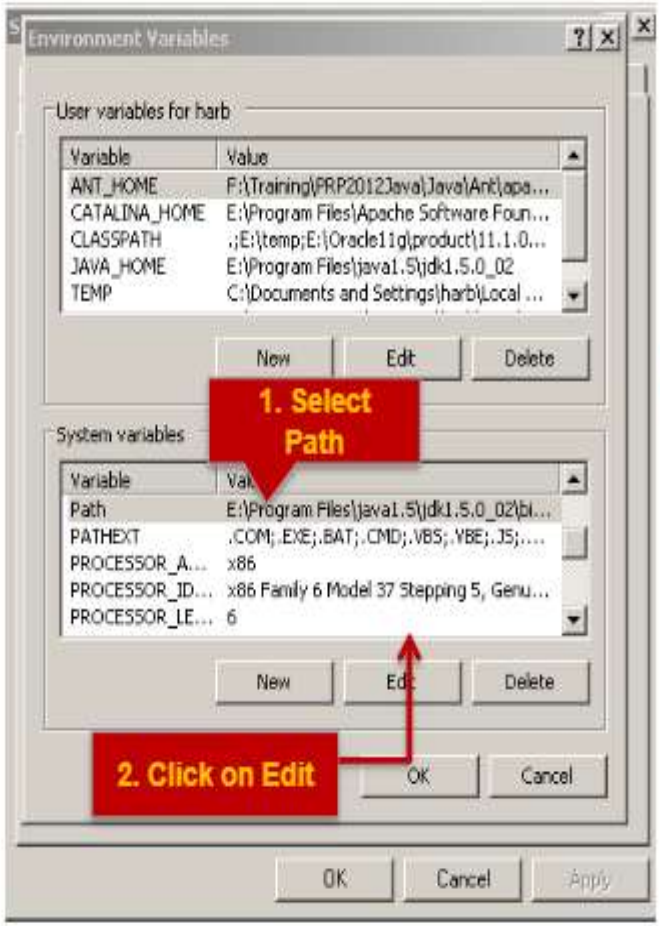
- We have to define PATH and CLASSPATH parameters and create necessary directories(folders) where we will be storing all our program files

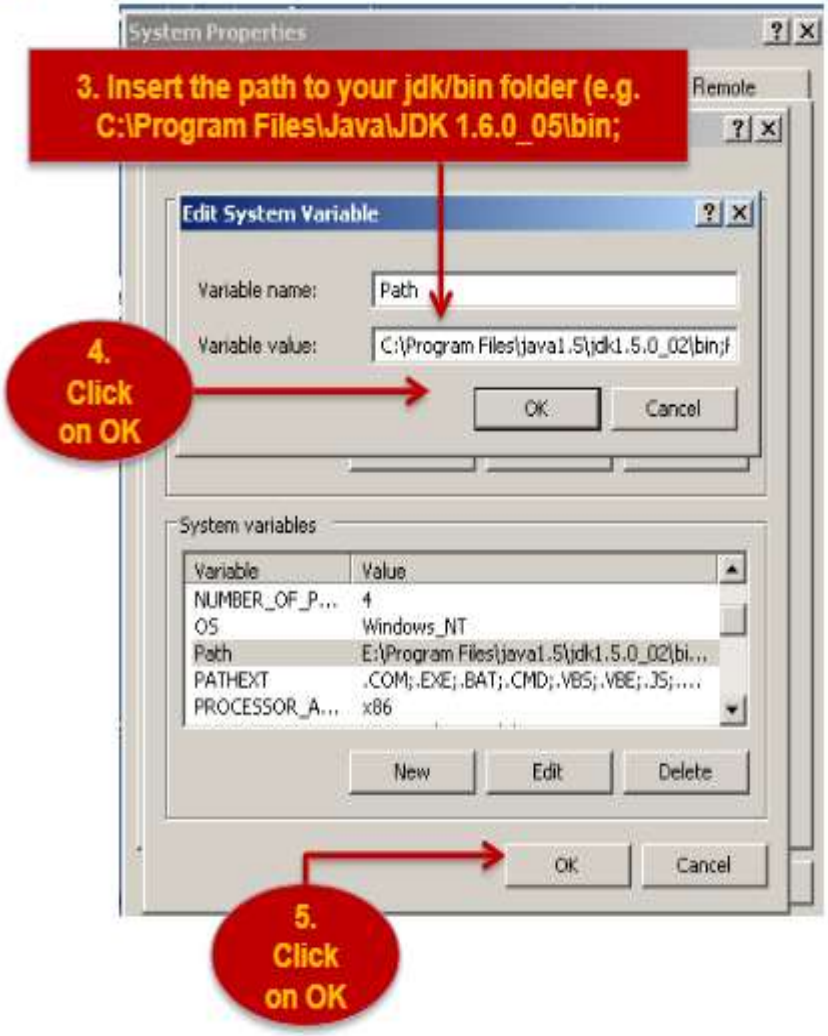
How to set PATH ?

1. Right Click My Computer
2. Select Properties
3. You will get to see the Properties Page of My Computer
4. Select Advanced Tab
5. Select Environment Variables
6. You will see Environment Variables Page as displayed here



How to set PATH ? (Contd.).



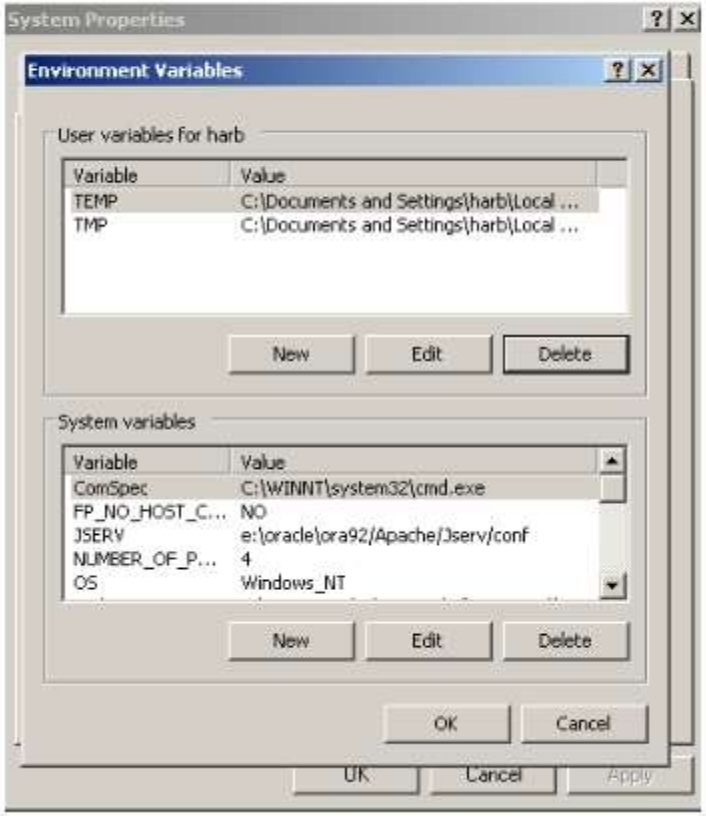


CLASSPATH

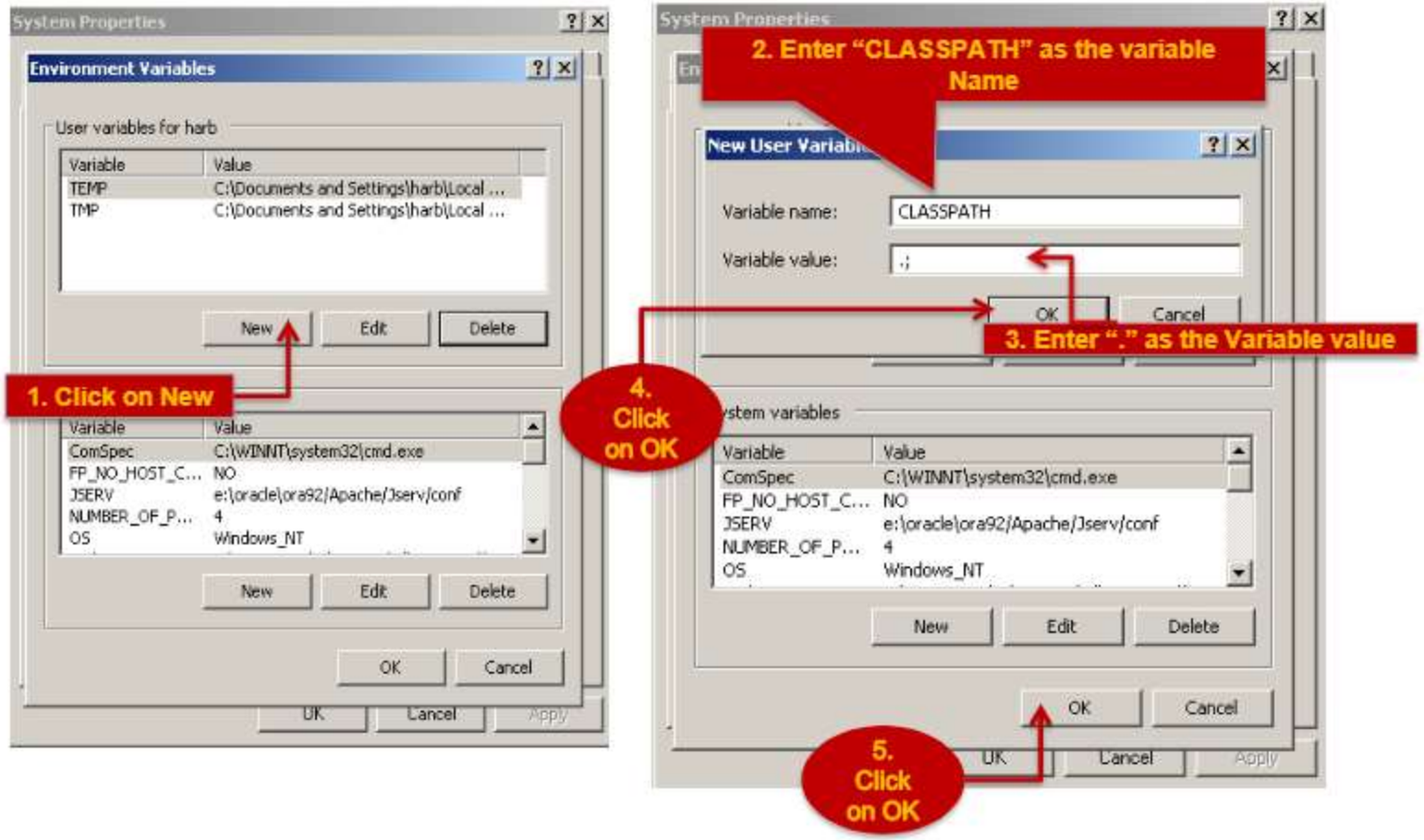
- CLASSPATH is a parameter which tells the JVM or the Compiler, where to locate classes that are not part of Java Development ToolKit(JDK).
- CLASSPATH is set either on command-line or through environment variable.
- CLASSPATH set on command-line is temporary in nature, while the environment variable is permanent.

How to set CLASSPATH ?

1. Right Click My Computer
2. Select Properties
3. You will get to see the Properties Page of My Computer
4. Select Advanced Tab
5. Select Environment Variables
6. You will see Environment Variables Page as displayed here



How to set CLASSPATH ? (Contd.).



Simple Hello World Program:

```
import java.io.*;

class Simple
{
    public static void main(String arg[])
    {
        System.out.println("Hello Java");
    }
}
```

Parameters used in first java program

class - keyword is used to declare a class in java.

public - keyword is an access modifier which represents visibility, it means it is visible to all.

Static - is a keyword, if we declare any method as static

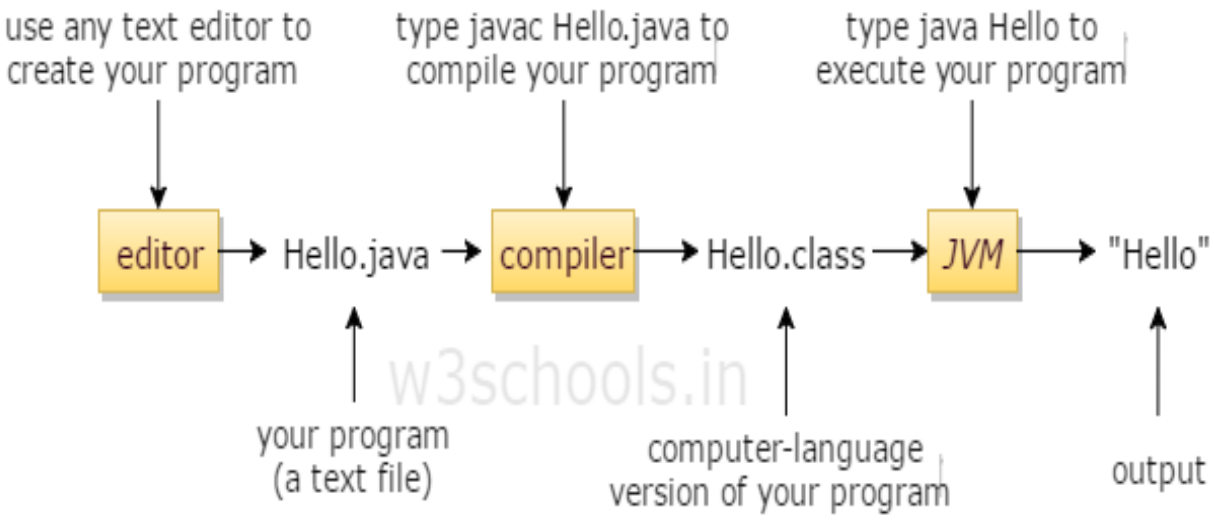
Void - is the return type of the method, it means it doesn't return any value.

Main - represents the starting point of the program.

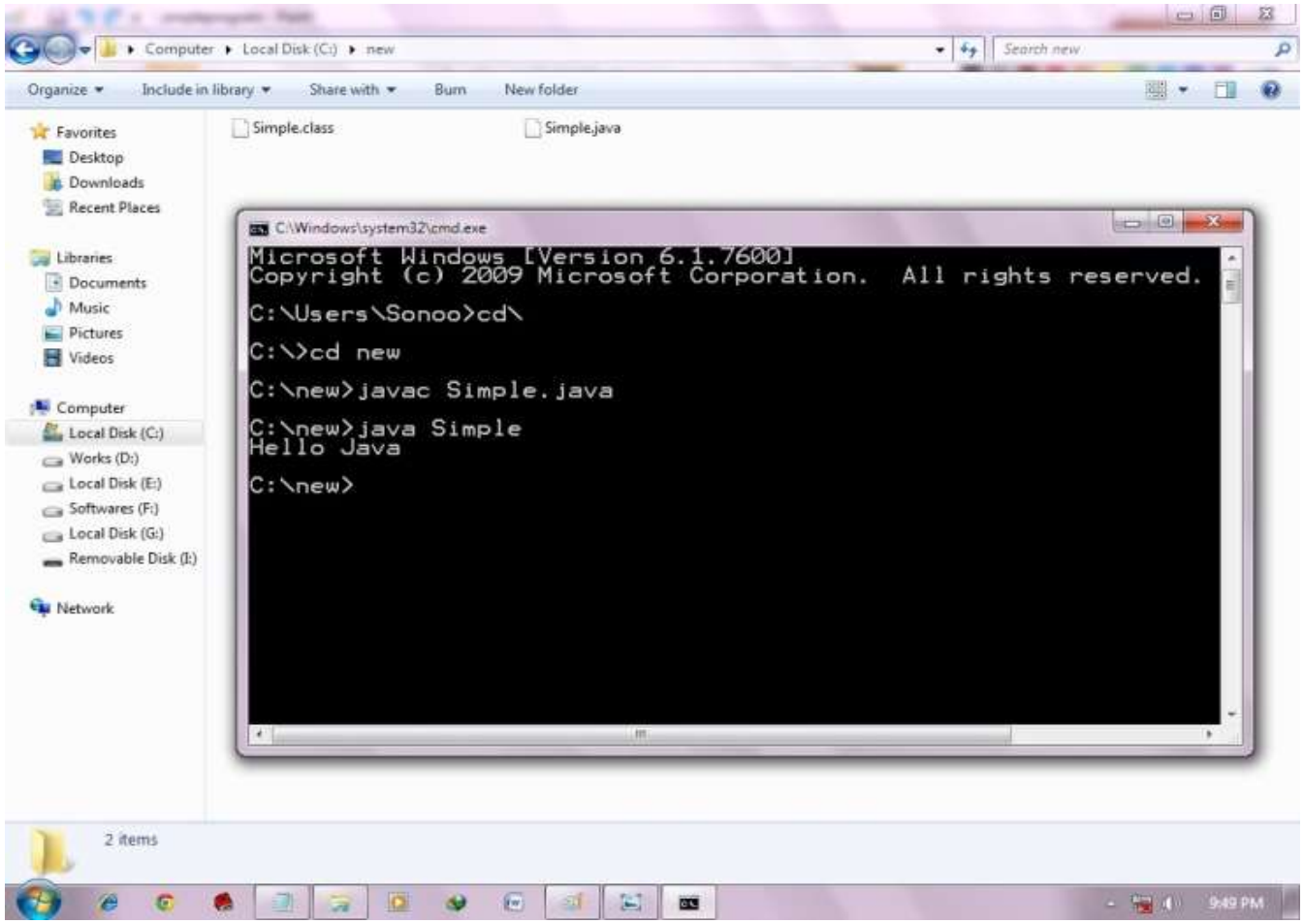
String[] args - is used for command line argument.

System.out.println() - is used print statement.

How to compile a java Program



Editing, compiling and executing.


Computer ▸ Local Disk (C:) ▸ new
Search new
Organize
Include in library
Share with
Burn
New folder
Favorites
Desktop
Downloads
Recent Places
Libraries
Documents
Music
Pictures
Videos
Computer
Local Disk (C:)
Works (D:)
Local Disk (E:)
Softwares (F:)
Local Disk (G:)
Removable Disk (I:)
Network
Simple.class
Simple.java
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
C:\Users\Sonoo>cd\
C:\>cd new
C:\new>javac Simple.java
C:\new>java Simple
Hello Java
C:\new>
2 items
9:49 PM

Language Basics

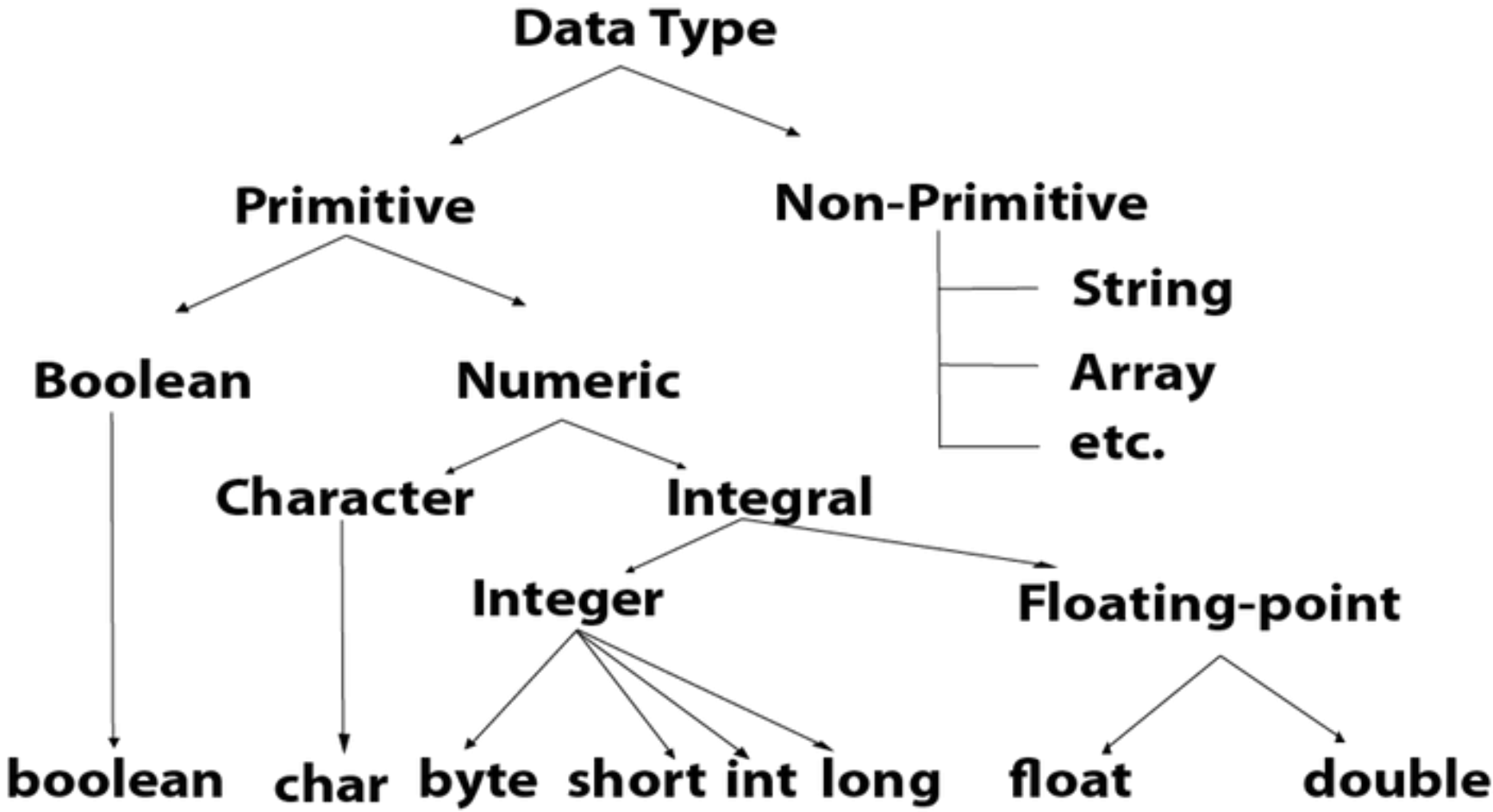
- Keywords
- Data Types
- Variables
- Operators
- Conditional Statements
- Loops

Java Keywords

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

Data Types in Java

- Data types specify the different sizes and values that can be stored in the variable.
- There are two types of data types in Java:
  - **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
  - **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.


Data Type
Primitive
Non-Primitive
String
Array
etc.
Boolean
Numeric
Character
Integral
Integer
Floating-point
boolean
char
byte
short
int
long
float
double

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Primitive Data Types

Data Type	Size (in bits)	Minimum Range	Maximum Range	Default Value (for fields)
byte	8	-128	+127	0
short	16	-32768	+32767	0
int	32	-2147483648	+2147483647	0
long	64	-9223372036854775808	+9223372036854775807	0L
float	32	1.40E-45	3.40282346638528860e+38	0.0f
double	64	4.94065645841246544e-324d	1.79769313486231570e+308d	0.0d
char	16		0 to 65,535	'\u0000'
boolean	1	NA	NA	false

OPERATOR

- An operator is a symbol that operates on one or more arguments to produce a result.

- Java provides a rich set of operators to manipulate variables.

TYPES OF OPERATORS

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment and decrement operators
- Conditional operators
- Bitwise operators
- Special operators

Arithmetic Operators

Operator	Description	Example **a = 10, b = 2, c = 0**
+	Addition	c = a + b c = 10 + 2 c = 12
-	Subtraction	c = a - b c = 10 - 2 c = 8
*	Multiplication	c = a * b c = 10 * 2 c = 20
/	Division	c = a / b c = 10 / 2 c = 5
%	Modulus	c = a % b c = 10 % 2 c = 0

```
class OperatorExample
{
    public static void main(String args[])
    {
        int a=10;
        int b=5;
        System.out.println(a+b);     //15
        System.out.println(a-b);     //5
        System.out.println(a*b);     //50
        System.out.println(a/b);     //2
        System.out.println(a%b);     //0
    }
}
```

Relational Operators

Operator	Description	Example
==	Check Equal	a == 5 5 == 5
!=	Check Not Equal	a != 5 6 != 5
>	Check Greaterthan	a > 5 10 > 5
<	Check Lessthan	a < 5 3 < 5
>=	Check Greaterthan or Equal	a >= 5 6 >= 5
<=	Check Lessthan or Equal	a <= 5 4 <= 5

```
public class Operator_Relation
{
    int a = 10;
    void relation() {
            if(a==10) {
                    System.out.println("Equal Operator : Verified");
            }
            if(a!=20) {
                    System.out.println("Not Equal Operator : Verified");
            }
            if(a>5) {

                    System.out.println("Greaterthan Operator : Verified");
            }
            if(a<15) {
                    System.out.println("Lessthan Operator : Verified");
            }
            if(a>=9) {
                    System.out.println("Greaterthan or Equal Operator : Verified");
            }
            if(a<=11) {
                    System.out.println("Lessthan or Equal Operator : Verified");
            }
    }
}
 class MainClass {
    public static void main (String args[]) {
            Operator_Relation obj = new Operator_Relation();
            obj.relation();
    }
}
```

Logical Operators:

Operator	Description	Example
&&	Logical AND Operator	a && b
||	Logical OR Operator	a || b
!	Logical NOT Operator	!(a && b)

```
public class Operator_Logical
{
   public static void main(String args[])
   {
         boolean a = true;
         boolean b = false;
         boolean c;
         System.out.println("Logical Operators");
         c = a && b;
         System.out.println("\n  a && b :" + c);
         c = a || b;
         System.out.println("a || b : " + c);
         c = !(a && b);
         System.out.println("\n  ! (a && b) : " + c);
         c = !(a || b);
         System.out.println("!(a || b) : " + c);
   }
}
```

```
Logical Operators
a && b :false
a || b : true
!(a && b) : true
!(a || b) : false
```

Assignment Operators:

Operator	Example	Equivalent Expression
=	z = x + y	z = (x + y)
+=	x += y	x = (x + y)
-=	x -= y	x = (x - y)
*=	x *= y	x = (x * y)
/=	x /= y	x = (x / y)

```
class OperatorExample
{
    public static void main(String[] args)
    {
        int a=10;
        a+=3;                    //a=a+3 ➔ a=10+3
        System.out.println(a);
        a-=4;                    //a=a-4 ➔ a=13-4
        System.out.println(a);
        a*=2;                    //a=a*2 ➔ a=9*2
        System.out.println(a);
        a/=2;                    //a=a/2 ➔ a=18/2
        System.out.println(a);
    }
}
```

Increment and Decrement

- ✓ The increment and decrement operators are arithmetic and operate on one operand
- ✓ The *increment operator* (++) adds one to its operand
- ✓ The *decrement operator* (--) subtracts one from its operand
- ✓ The statement count++;

is functionally equivalent to count = count + 1;

Increment and Decrement

- ✓ When used in a larger expression, the prefix and postfix forms have different effects
- ✓ In both cases the variable is incremented (decremented)
- ✓ But the value used in the larger expression depends on the form used.
- ✓ If count currently contains 45, then the statement

```
total = count++;
```

assigns 45 to total and 46 to count

- ✓ If count currently contains 45, then the statement

```
total = ++count;
```

assigns the value 46 to both total and count

Increment And Decrement Operators:

- **Increment operators:**
  - Pre-increment(++m)
  - Post increment(m++)
- **Decrement operators:**
  - Pre-decrement(--m)
  - Post-decrement

		c = ++a
++	**Increment**	**c = 10 + 1**
		c = 11
		c = --a
- -	**Decrement**	**c = 11 - 1**
		c = 10

```
class OperatorExample
{
        public static void main(String args[])
        {
            int x=10;
            System.out.println(x++);                //10 (11)
            System.out.println(++x);                //12
            System.out.println(x--);        //12 (11)
            System.out.println(--x);        //10
        }
}
```

```
class OperatorExample
{
    public static void main(String args[])
    {
        int a=10;
        int b=10;
        System.out.println(a++ + ++a);
        System.out.println(b++ + b++);
    }
}
```

Conditional Operators:

- The character pair **? :** is a ternary operator available in Java.
- This is used to condition expression of the form:

```
exp ?  exp2  : exp3
```

```
class OperatorExample
{
  public static void main(String args[])
  {
      int a=2;
      int b=5;
      int min=(a<b)?a:b;
      System.out.println(min);
  }
}
```

Bitwise Operators:

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise Exclusive OR
~	One's complement
<<	Shift left
>>	Shift right
>>>	Shift right with zero fill

Bitwise Operator

- ✓ Applied to integer type – long, int, short, byte and char.

A	B	A | B	A & B	A^B	~A
0	0	0	0	0	1
0	1	1	0	1	1
1	0	1	0	1	0
1	1	1	1	0	0

Example

```
public class BitwiseAndExample
{
 public static void main(String[] args)
 {
   int x = 9, y = 8;
      // bitwise and
      // 1001 & 1000 = 1000 = 8
  System.out.println("x & y = " + (x & y));
 }
}
```

```
public class BitwiseXorExample
{
 public static void main(String[] args)
 {
   int x = 9, y = 8;
      // bitwise XOR
      // 1001 ^ 1000 = 0001 = 1
   System.out.println("x ^ y = " + (x ^ y));
 }
}
```

```
public class LeftShiftExample
{
 public static void main(String args[])
 {
   int x = 12;    // 1100
  System.out.println("x<<1 = " + (x << 1));
 }
}
```

```
public class RightShiftExample
{
 public static void main(String args[])
 {
   int x = 50;    // 0011 0010
   System.out.println("x>>2 = " + (x >>2));
 }
}
```

How to get input from user in Java

Java Scanner Class

- Java **Scanner class** allows the user to take input from the console.
- It belongs to **java.util** package.

Syntax:

```
Scanner sc=new Scanner(System.in);
```

Methods of Java Scanner Class

Method	Description
int nextInt()	It is used to scan the next token of the input as an integer.
float nextFloat()	It is used to scan the next token of the input as a float.
double nextDouble()	It is used to scan the next token of the input as a double.
byte nextByte()	It is used to scan the next token of the input as a byte.
String nextLine()	Advances this scanner past the current line.
boolean nextBoolean()	It is used to scan the next token of the input into a boolean value.
long nextLong()	It is used to scan the next token of the input as a long.
short nextShort()	It is used to scan the next token of the input as a Short.
BigInteger nextBigInteger()	It is used to scan the next token of the input as a BigInteger.
BigDecimal nextBigDecimal()	It is used to scan the next token of the input as a BigDecimal.

```
import java.util.*;
class UserInputDemo
{
    public static void main(String[] args)
    {
      Scanner sc= new Scanner(System.in);    //System.in is a standard input stream
      System.out.print("Enter first number- ");
       int a= sc.nextInt();
      System.out.print("Enter second number- ");
      int b= sc.nextInt();
      System.out.print("Enter third number- ");
      int c= sc.nextInt();
     int d=a+b+c;
     System.out.println("Total= " +d);
    }
}
```

Example of String Input from user

```
import java.util.*;
class UserInputDemo1
{
    public static void main(String[] args)
    {
        Scanner sc= new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str= sc.nextLine();          //reads string
        System.out.print("You have entered: "+str);
    }
}
```

Control Statements in Java

- Java language possesses such decision making capabilities and supports the following statements known as control or decision statements.

  - **Selection/Decision Making Statements**
  - **Iteration/Looping Statements**
  - **Jump Statements**

Decision Making Statements

Sr.No.	Statement & Description
1	**if statement** An **if statement** consists of a boolean expression followed by one or more statements.
2	**if...else statement** An **if statement** can be followed by an optional **else statement**, which executes when the boolean expression is false.
3	**nested if statement** You can use one **if** or **else if** statement inside another **if** or **else if** statement(s).
4	**switch statement** A **switch** statement allows a variable to be tested for equality against a list of values.

If Statement:

- The “if” is a powerful decision making statement and is used to control the flow of execution of statements.
- It is basically a two-way decision statement and is used with the conjunction with an expression.
- It takes the following form:

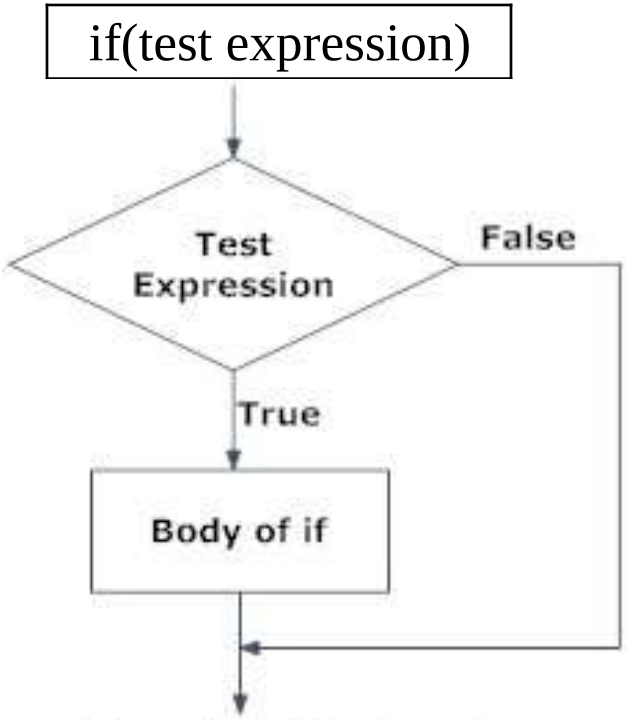


Fig: Operation of if statement

Simple If Statement:

- The general form of a simple if statement is:

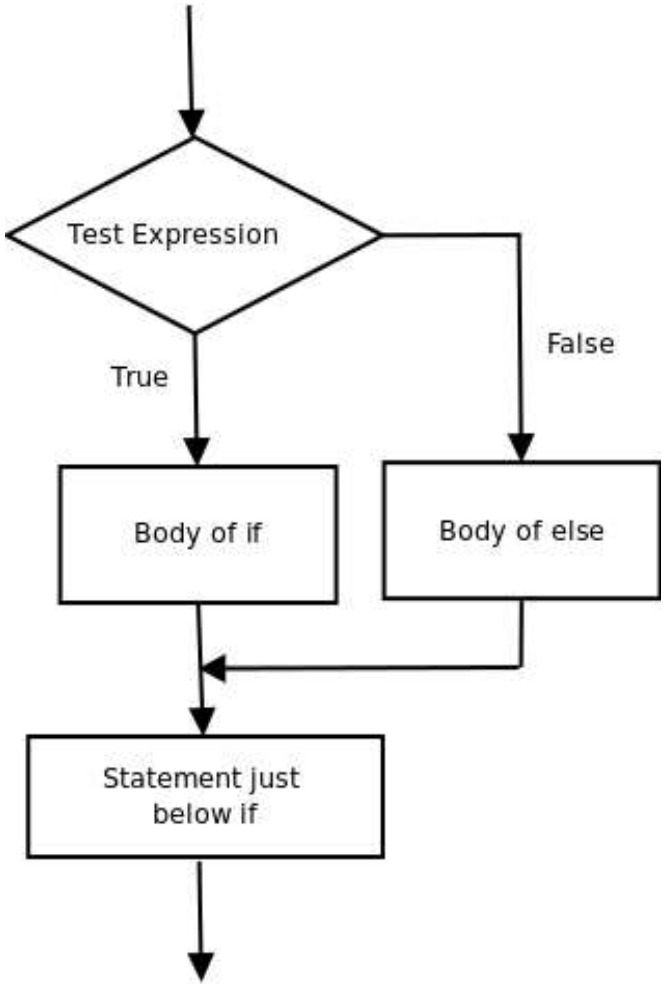
```
if(test expression)
{
Statement block;
}
Statement-X
```

If-Else Statement:

- The if-else statement is the extension of the simple **if** statement.
- The general form is,

```
if(test expression)
{
    True statement;
}
else
{
    False statement
}
statement X;
```

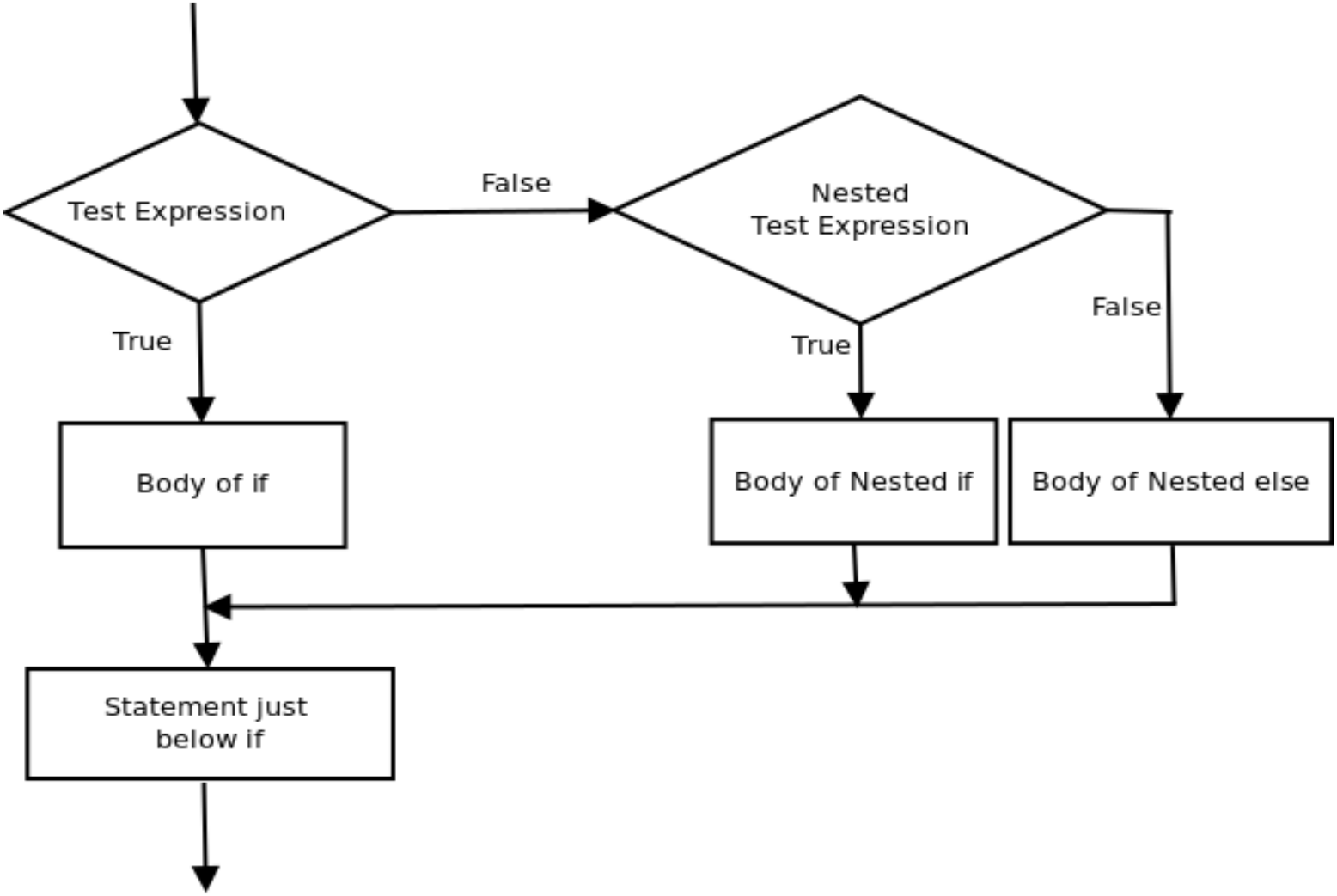
```
public class IfElseExample {
    public static void main(String[] args) {

   int number=13;
    if(number%2==0)
   {
      System.out.println("even number");
   }
else     {
      System.out.println("odd number");
   }
System.out.println("executed Successfully");
}
}
```



Nesting of If…Else Statements:

- When a series of decisions are involved, we may have to use more than one **if…else** statement in nested as follows:

```
if(test condition1)
{
        if( test condition2)
         {
                Statement 1;
         }
         else
          {
                Statement 2;
         }
 }
 else
 {
        Statement 3;
  }
   Statement X;
```

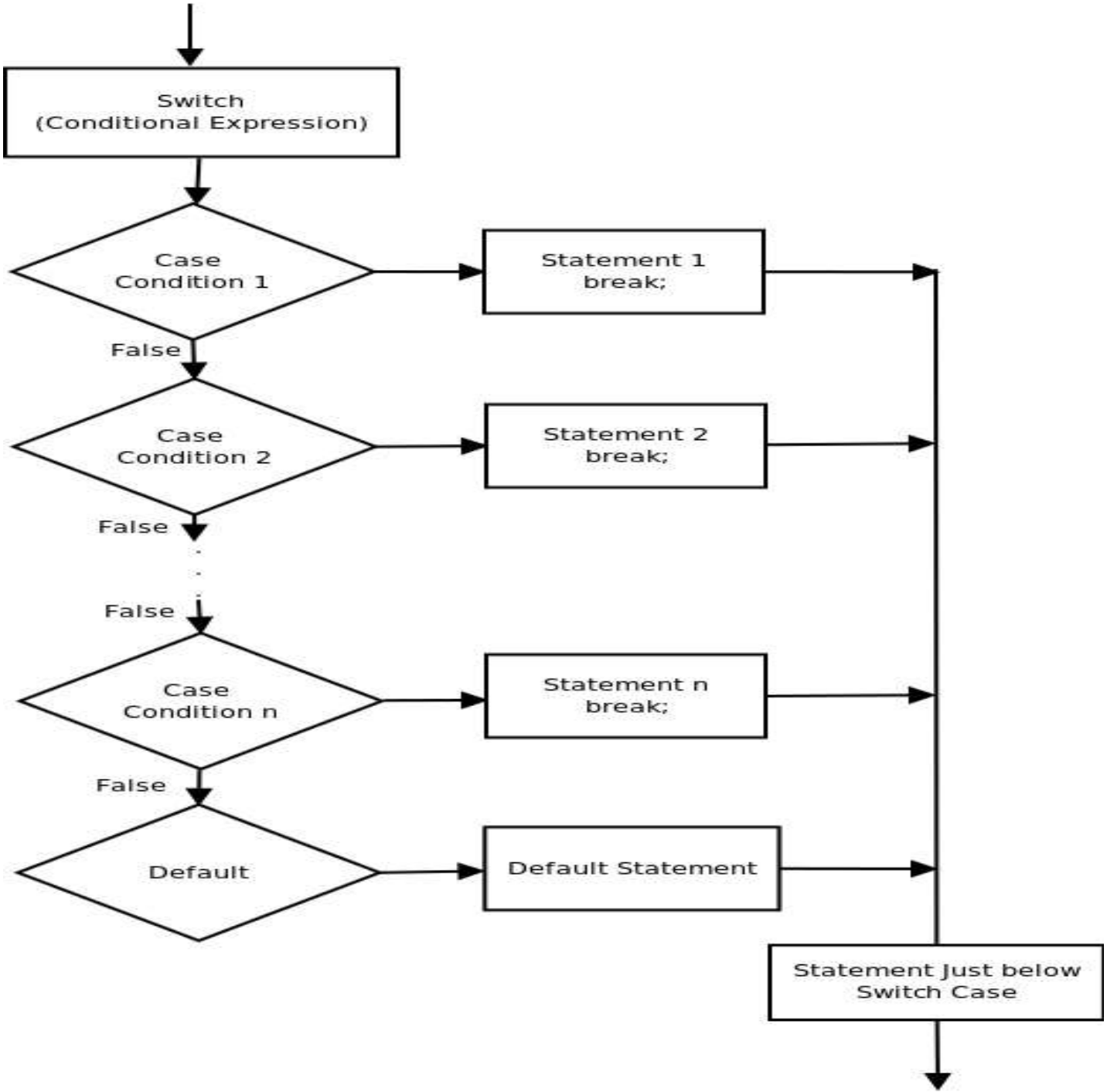

Test Expression
False
Nested
Test Expression
True
True
False
Body of if
Body of Nested if
Body of Nested else
Statement just
below if

```
public class PositiveNegativeExample
 {
            public static void main(String[] args)
            {
              int number=-13;
             if(number>0)
            {
             System.out.println("POSITIVE");
             }
            else if(number<0)
            {
            System.out.println("NEGATIVE");
            }
            else
            {
             System.out.println("ZERO");
            }
   }
}
```

The switch statement:

- We have seen that one of the many alternatives is to be selected, we can design a program using **if** statements to control the selection.
- Java has a built-in multiway decision statement known as **switch.**
- The general form of switch is given below:

```
switch(expression)
{
  case 1:        block-1
                 break;
  case 2:
                 block-2;
                 break;
  . . . . .
  . . . . .
  default:
                 default-block;
}
Statement-X;
```


Switch
(Conditional Expression)
Case
Condition 1
Statement 1
break;
False
Case
Condition 2
Statement 2
break;
False
False
Case
Condition n
Statement n
break;
False
Default
Default Statement
Statement Just below
Switch Case

```
public class SwitchExample
 {
public static void main(String[] args)
{
      int number=20;
      switch(number){
       case 10: System.out.println("10");
        break;
        case 20: System.out.println("20");
        break;
      case 30: System.out.println("30");
        break;
   //Default case statement
   default:
     System.out.println("Not in 10, 20 or 30");
   }
}
}
```

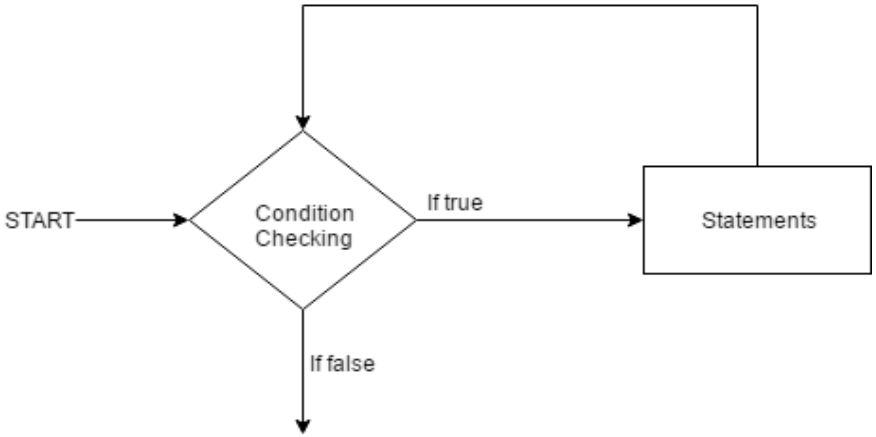
Iteration/Looping Statements

- Repeating the same code fragment several times until a specified condition is satisfied is called iteration.
- Iteration statements execute the same set of instructions until a termination condition is met.
- Java provides the following loop for iteration statements:
    - The while loop
    - The for loop
    - The do-while loop
    - The for each loop

The while loop

- The simplest of all the looping structures in Java is the **while** statement.
- The 'while' is a **entry controlled** loop statement. The **test condition** is evaluated first, if it is true then the body of the loop is executed.
- The basic format of the while statement is:

```
initialization
while( test condition)
{
        Body of the loop;
}
```

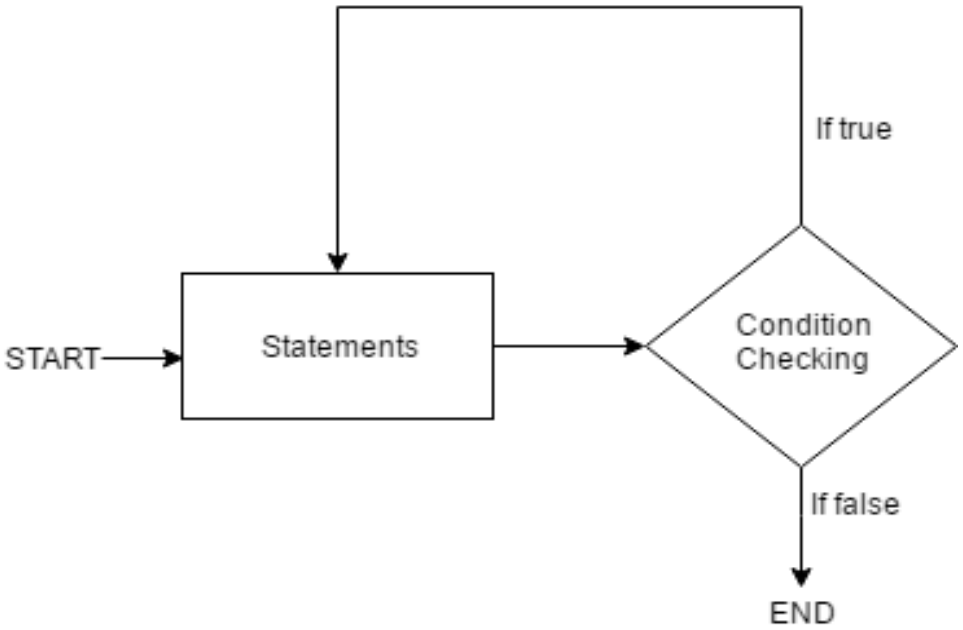


```
public class WhileDemo
{
    public static void main( String[] args )
    {
        int i = 0;
        while ( i < 5 )
        {
            System.out.println( "Value :: " + i );
            i++;
        }
    }
}
```

The do-While Statement:

- The only difference between a while and a do-while loop is that do-while evaluates its expression at the bottom of the loop instead of the top.
- The do-while loop executes at least one time then it will check the expression prior to the next iteration.

```
initialization
do
{
Body of the loop
}while( test condition)
```



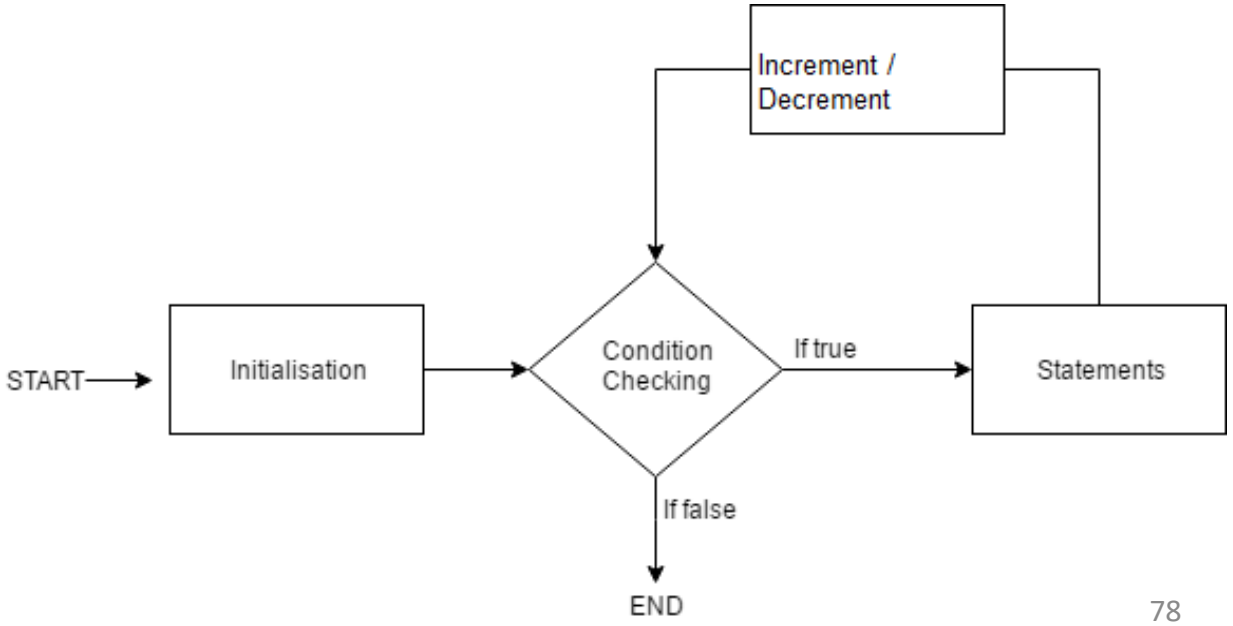
```
public class DoWhileDemo
{
    public static void main( String[] args )
    {
        int i = 0;
        do
        {
            System.out.println( "value :: " + i );
            i++;
        }
        while ( i < 5);
    }
}
```

while	do while
It is an entry controlled loop	It is an exit controlled loop
It is a loop that will execute only if the condition is true	It is a loop that execute atleast once even if the condition is false
It is generally used for implementing common looping situations	It is typically used for implementing menu based programs where the menu is required to be printed atleast once

The for loop

- A for loop executes a statement (that is usually a block) as long as the boolean condition evaluates to true.
- A for loop is a combination of the three elements initialization statement, boolean expression and increment or decrement statement.

```
for( initialization ;  test condition ; increment/decrement)
 {
            //Body of the loop
 }
```



```
import java.io.*;
public class ForDemo
{
   public static void main( String[] args )
   {
      for(int i = 0;  i<5;i++)
     {
         System.out.println( "Value :: " + i );
      }
   }
}
```

The For each loop

- This was introduced in Java 5.
- This loop is basically used to traverse the array or collection elements.

Syntax:

```
for (type var : array)
{
        statements using var;
}
```

is equivalent to:

```
for (int i=0; i<arr.length; i++)
{
        type var = arr[i];
        statements using var;
}
```

```
import java.io.*;
public class ForEachDemo
{
   public static void main( String[] args )
   {
      int[] i = { 1, 2, 3, 4, 5 };
      for ( int j : i )
      {
         System.out.println( "value :: " + j );
      }
   }
}
```

JUMP STATEMENTS:

- Jump statements are used to interrupt the normal flow of program.

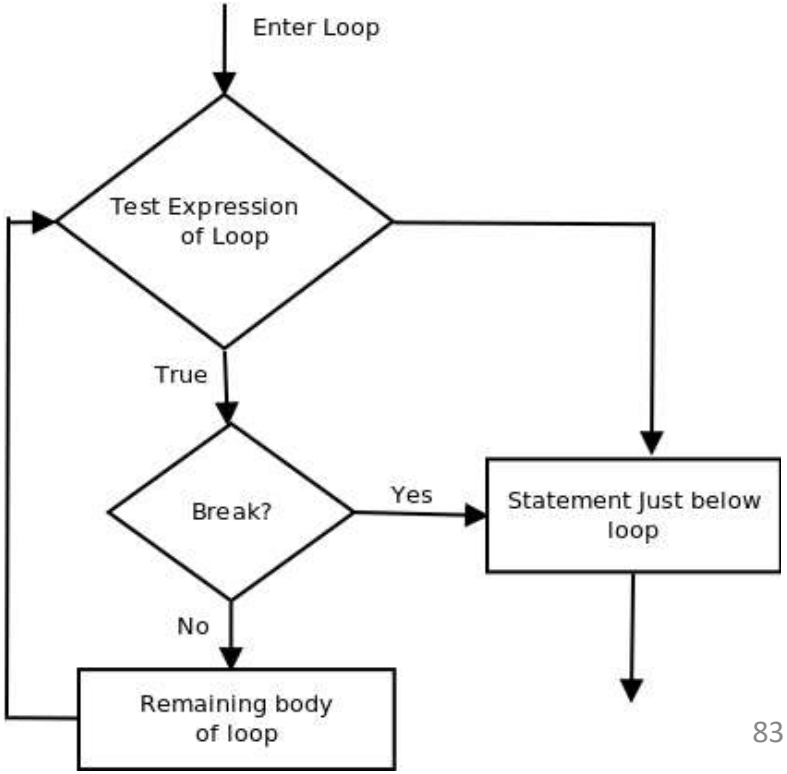
- Types of Jump Statements are:
    - break
    - continue
    - return

Break Statement

- break statement is used inside loop or switch statement.
- When compiler finds the break statement inside a loop, compiler will abort the loop and continue to execute statements followed by loop.

Syntax:-

```
break;
```



```
class BreakStatementDemo
  {
    public static void main(String args[])
    {
      int a=1;
      while(a<=10)
      {
        if(a==5)
          break;
        System.out.print("\n\tStatement : " + a);
        a++;
      }
      System.out.print("\n\tEnd of Program.");
    }
  }
```

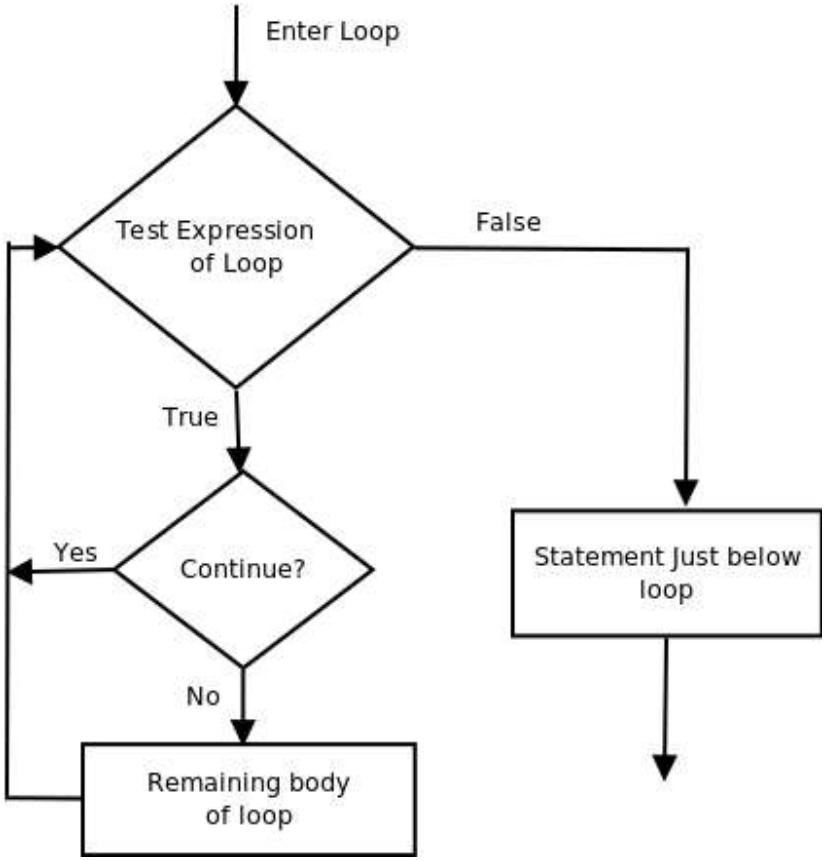
```
Statement : 1
Statement : 2
Statement : 3
Statement : 4
End of Program.
```

Continue Statement

- The continue statement is also used inside loop.
- The continue statement is used when you want to continue running the loop with the next iteration and want to skip the rest of the statements of the body for the current iteration.

Syntax:-

```
continue;
```



```
class ContinueStatementDemo
  {
    public static void main(String args[])
    {
      int a=0;
      while(a<5)
      {
        a++;
        if(a==5)
          continue;
        System.out.print("\n\tStatement " + a);
      }
      System.out.print("\n\tEnd of Program.");
  }
```

Return Statement

- The return statement is used to immediately quit the current method and return to the calling method.
- It is mandatory to use a return statement for non-void methods to return a value.

Syntax:-

```
return;
```

```
import java.io.*;
public class ReturnDemo
{
   public static void main( String[] args )
   {
      ReturnDemo returnDemo = new ReturnDemo();
      System.out.println( "No : " + returnDemo.returnCall() );
   }

   int returnCall()
   {
      return 5;
   }
}
```

Command Line Arugement

- The java command-line argument is an argument i.e. passed at the time of running the java program.

```
class CommandLineExample
{
    public static void main(String args[])
    {
        System.out.println("Your first argument is: "+args[0]);
    }
}
```

```
class A{
        public static void main(String args[])
        {

        for(int i=0;i<args.length;i++)
        System.out.println(args[i]);

        }
}
```

Update Completion Status

No.	Hands-on Assignment	Topics Covered
1	Write a Program that accepts two Strings as command line arguments and generate the output in the required format. Example1) If the two command line arguments are Wipro and Bangalore then the output generated should be Wipro Technologies Bangalore. Example2) If the command line arguments are ABC and Mumbai then the output generated should be ABC Technologies Mumbai [Note: It is mandatory to pass two arguments in command line]	Command Line Argument
2	Write a Program to accept a String as a command line argument and print a Welcome message as given below. Example1) C:\> java Sample John O/P Expected : Welcome John	Command Line Argument
3	Write a Program to accept two integers as command line arguments and print the sum of the two numbers Example1) C:\>java Sample 10 20 O/P Expected : The sum of 10 and 20 is 30	Command Line Argument

Update Completion Status

No.	Hands-on Assignment	Topics Covered
1	A) Write a program to check if a given integer number is Positive, Negative, or Zero. B) Given two non-negative int values, print true if they have the same last digit, such as with 27 and 57. lastDigit(7, 17) → true lastDigit(6, 17) → false lastDigit(3, 113) → true	If Statement
2	Write a program to check if a given integer number is odd or even.	If Statement
3	Write a program to check if the program has received command line arguments or not. If the program has not received arguments then print "No Values", else print all the values in a single line separated by ,(comma) Example1) java Example O/P: No values Example2) java Example Mumbai Bangalore O/P: Mumbai,Bangalore [Hint: You can use length property of an array to check its length]	If Statement
4	Initialize two character variables in a program and display the characters in alphabetical order. Example1) if the first character is 's' and second character is 'e' then the output should be e,s Example2) if the first character is 'a' and second character is 'e', then the output should be a,e	If Statement

6	Write a program to accept gender ("Male" or "Female") and age from command line arguments and print the percentage of interest based on the given conditions. If the gender is 'Female' and age is between 1 and 58, the percentage of interest is 8.2%. If the gender is 'Female' and age is between 59 and 100, the percentage of interest is 9.2%. If the gender is 'Male' and age is between 1 and 58, the percentage of interest is 8.4%. If the gender is 'Male' and age is between 59 and 100, the percentage of interest is 10.5%.	If Statement
7	Initialize a character variable with an alphabhet in a program. If the character value is in lowercase, the output should be displayed in uppercase in the following format. Example1) i/p:a o/p:a->A If the character value is in uppercase, the output should be displayed in lowercase in the following format. Example2) i/p:A o/p:A->a	If Statement
8	Write a program to receive a color code from the user (an Alphabhet). The program should then print the color name, based on the color code given. The following are the color codes and their corresponding color names. R->Red, B->Blue, G->Green, O->Orange, Y->Yellow, W->White. If color code provided by the user is not valid then print "Invalid Code".	Switch Statement

8	Write a program to receive a color code from the user (an Alphabhet). The program should then print the color name, based on the color code given. The following are the color codes and their corresponding color names. R->Red, B->Blue, G->Green, O->Orange, Y->Yellow, W->White. If color code provided by the user is not valid then print "Invalid Code".	Switch Statement
9	Write a program to receive a number and print the corresponding month name. Example1) C:\>java Sample 12 O/P Expected : December Example2) C:\>java Sample O/P Expected : Please enter the month in numbers Example3) C:\>java Sample 15 O/P Expected : Invalid month	Switch Statement
10	Write a program to print numbers from 1 to 10 in a single row with one tab space.	For Loop
11	Write a program to print even numbers between 23 and 57. Each number should be printed in a separate row.	For Loop

14	Write a program to print the sum of all the digits of a given number. Example1) I/P:1234 O/P:10	For Loop	☐
15	Write a program to print * in Floyds format (using for and while loop) * * * * * * Example1) C:\>java Sample O/P: Please enter an integer number Example2) C:\>java Sample 3 O/P : * * * * * *	For Loop	☐
16	Write a program to reverse a given number and print Example1) I/P: 1234 O/P:4321 Example2) I/P:1004 O/P:4001	While Loop	☐

Sample Questions

After gaining knowledge of the above module, below are the possible interview questions that you should be able to confidently answer.

No	Questions
1	What is JDK?
2	What is JRE?
3	What is JIT compiler?
4	What is Adaptive optimizer?
5	What is Class Loader?
6	What is Byte Code Verifier?
7	What is a Compiler?
8	What is an Interpreter?
9	What is the difference between a Compiler and Interpreter?
10	What are the components of JVM?
11	What is Platform Independent?
12	How does Java achieve Platform independence?
13	How to access command line arguments?
14	What are the data types in Java and what are their sizes?
15	How do you declare an array in Java?
16	How do you find the length of an array in Java?
17	What is the use of PATH variable?
18	What is the use of CLASSPATH variable?
19	How do you assign values for PATH and CLASSPATH Variable?
20	What are all the characteristic features of Java Programming language?

Example 3:

```
public class jj {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int sum = a + b;
        System.out.println("Sum is " + sum);
    }
}
```

Example - 5

```
import java.util.Scanner;
public class AlphabetOrder {
        public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
                char c1=s.next().charAt(0);
                char c2=s.next().charAt(0);
                if(c1>c2) {
                System.out.println(c2+","+c1);

                }
                else {
                        System.out.println(c1+","+c2);
                }
        }
}
```

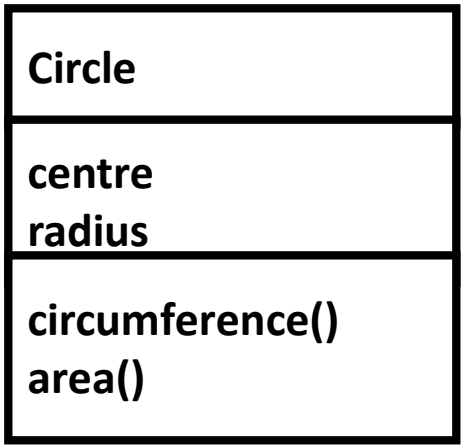
Example - 6

```
public class GenderInterest {

public static void main(String[] args)
 {
String gender=args[0];
int age=Integer.parseInt(args[1]);
if(gender.equalsIgnoreCase("Female")) {
  if(age>=1&&age<=58)
      System.out.println("Interest=8.2%");
else if(age>58&&age<=100)
   System.out.println("Interest=9.2%");
else
System.out.println("No result");
}
else {
      if(age>=1&&age<=58)
     System.out.println("Interest=8.4%");
      else if(age>58&&age<=100)
      System.out.println("Interest=10.5%");
  else
  System.out.println("No result");
}
}
}
```

Class & Objects in Java

Class

- A class is a collection of fields (data) and methods (procedure or function) that operate on that data.



Class

- A *class* is a collection of *fields* (data) and *methods* (procedure or function) that operate on that data.
- The basic syntax for a class definition:

```
class  ClassName [extends
SuperClassName]
{
        [fields declaration]
        [methods declaration]
}
```

- Bare bone class – no fields, no methods

```
public class Circle {
    // my circle class
}
```

Adding Fields: Class Circle with fields

- Add *fields*

```
public class Circle {
      public double x, y;  // centre coordinate
      public double r;     //  radius of the circle

}
```

- The fields (data) are also called the *instance* variables.

Adding Methods

- A class with only data fields has no life. Objects created by such a class cannot respond to any messages.
- Methods are declared inside the body of the class but immediately after the declaration of data fields.
- The general form of a method declaration is:

```
type MethodName (parameter-list)
{
        Method-body;
}
```

Adding Methods to Class Circle

```
public class Circle {

    public double x, y; // centre of the circle
    public double r;    // radius of circle

    //Methods to return circumference and area
    public double circumference() {
            return 2*3.14*r;
    }
    public double area() {
            return 3.14 * r * r;
    }
}
```

Method Body

Data Abstraction

- Declare the Circle class, have created a new data type – Data Abstraction

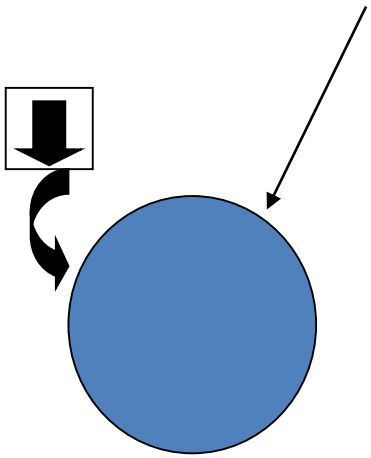
- Can define variables (objects) of that type:

```
Circle  aCircle;
Circle  bCircle;
```

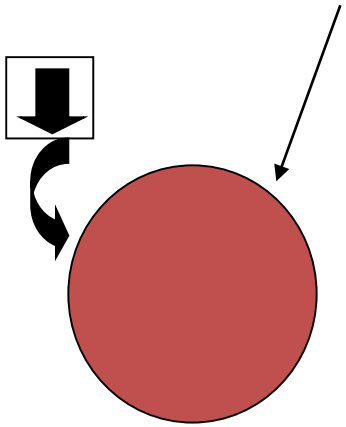
Creating objects of a class

- Objects are created dynamically using the *new* keyword.
- aCircle and bCircle refer to Circle objects

aCircle = new Circle() ;



bCircle = new Circle() ;



3 Ways to initialize object

- By reference variable
- By method
- By constructor

Object and Class Example: Initialization through reference

```
class Student{
    int id;
    String name;
}
class TestStudent2{
    public static void main(String args[]){
            Student s1=new Student();
           s1.id=101;
           s1.name=“example object";
    System.out.println(s1.id+" "+s1.name);
 }
}
```

```
class Student{
        int id;
        String name;
}
class TestStudent3{
        public static void main(String args[]){
                //Creating objects
        Student s1=new Student();
        Student s2=new Student();
 //Initializing objects
        s1.id=101;
         s1.name=“Arun";
        s2.id=102;
         s2.name=“Shashini";
 //Printing data
         System.out.println(s1.id+" "+s1.name);
        System.out.println(s2.id+" "+s2.name);
 }
}
```

Object and Class Example: Initialization through method

```
class Student{
        int rollno;
        String name;
 void insertRecord(int r, String n){
                 rollno=r;
                name=n;
 }
 void displayInformation()
{
System.out.println(rollno+" "+name);}
}
class TestStudent4{
                public static void main(String args[]){
                Student s1=new Student();
                Student s2=new Student();
                s1.insertRecord(111,"Karan");
                s2.insertRecord(222,"Aryan");
                s1.displayInformation();
                s2.displayInformation();
 }
}
```

Example Program

```
public class Cube
{
    int length = 10;
    int breadth = 10;
    int height = 10;
    public int getVolume()
{
            return (length * breadth * height);
    }
    public static void main(String[] args)
    {
            Cube cubeObj;
            cubeObj = new Cube();
    System.out.println("Volume of Cube is : " +cubeObj.getVolume());

    }
}
```

Sample program

```
class Box {
double width;
double height;
double depth;
}
```

```
class BoxDemo {
public static void main(String args[])
{
Box mybox = new Box();
double vol;

mybox.width = 10;
mybox.height = 20;
mybox.depth = 15;

vol = mybox.width * mybox.height *
mybox.depth;
System.out.println("Volume is " + vol);
}
}
```

```
Volume is 3000.0
```

Sample program

```
class Box
{
double width;
double height;
double depth;

void volume()
{
System.out.print("Volume is ");
System.out.println(width * height * depth);
}
}
```

```
class BoxDemo3
{
public static void main(String args[]) {
Box mybox1 = new Box();
Box mybox2 = new Box();

mybox1.width = 10;
mybox1.height = 20;
mybox1.depth = 15;
mybox2.width = 3;
mybox2.height = 6;
mybox2.depth = 9;

mybox1.volume();

mybox2.volume();
}
}
```

```
Volume is 3000.0
Volume is 162.0
```

Constructor in Java

Constructor in Java

- A constructor is a block of codes similar to the method. It is called when an instance of the class is created.
- At the time of calling constructor, memory for the object is allocated in the memory.

Rules for creating Java constructor

- ✓ There are two rules defined for the constructor.
- ✓ Constructor name must be the same as its class name
- ✓ A Constructor must have no explicit return type
- ✓ A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors

- Default constructor (no-arg constructor)
- Parameterized constructor

Syntax:

```
class Test {
Test()
 { // constructor body }
 }
```

Bike is created

```
class Main {
private String name;
        Main()
        {
        System.out.println("Constructor Called:");
         name = "Programiz";
        }
public static void main(String[] args)
        {
        // constructor is invoked while // creating an object of the Main class
         Main obj = new Main();
        System.out.println("The name is " + obj.name);
        }
}
```

Bike is created

Java Default Constructor

- A constructor is called "Default Constructor" when it doesn't have any parameter.
- Syntax of default constructor:

```
<class_name>()
{
}
```

```
class Bike1
{
//creating a default constructor
        Bike1()
        {
        System.out.println("Bike is created");
        }
//main method
public static void main(String args[])
        {
        //calling a default constructor
        Bike1 b=new Bike1();
        }
}
```

OUTPUT:
Bike is Created

Java Parameterized Constructor

```
class Student4
{
   int id;
   String name;
   //creating a parameterized constructor
         Student4(int i,String n)
         {
            id = i;
            name = n;
            }
   //method to display the values
  void display()
{
System.out.println(id+" "+name);
}
      public static void main(String args[])
    {
   //creating objects and passing values
   Student4 s1 = new Student4(111,"Karan");
   Student4 s2 = new Student4(222,"Aryan");
   //calling method to display the values of object
   s1.display();
   s2.display();
  }
}
```

Output:
111 Karan
222 Aryan

Strings in JAVA

String

- String is basically an object that represents sequence of char values.
- An array of characters works same as java string.
  - For example:
    - char[] ch={'j','a','v','a',’‘,'p',‘r',’o',’g',’r',’a‘,’m’};
    - String s=new String(ch);
  - is same as:
    - String s="java program";

How to create String object?

- There are two ways to create String object:
    - By string literal
    - By new keyword

1) String Literal
   - Java String literal is created by using double quotes.
   - For Example:
     - String s="welcome";

2) By new keyword
   - String s=**new** String("Welcome");

```
public class StringExample
{
    public static void main(String args[])
    {
        String s1="java";
        char ch[]={'s','t','r','i','n','g','s'};
        String s2=new String(ch);
        String s3=new String("example");
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s3);
    }
}
```

Java String class methods

No.	Method	Description
1	char charAt(int index)	It returns char value for the particular index
2	int length()	It returns string length
3	static String format(String format, Object... args)	It returns a formatted string.
4	static String format(Locale l, String format, Object... args)	It returns formatted string with given locale.
5	String substring(int beginIndex)	It returns substring for given begin index.
6	String substring(int beginIndex, int endIndex)	It returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	It returns true or false after matching the sequence of char value.
8	static String join (CharSequence delimiter, CharSequence... elements)	It returns a joined string.
9	static String join (CharSequence delimiter, Iterable<? extends CharSequence> elements)	It returns a joined string.
10	boolean equals(Object another)	It checks the equality of string with the given object.

11	boolean isEmpty()	It checks if string is empty.
12	String concat(String str)	It concatenates the specified string.
13	String replace(char old, char new)	It replaces all occurrences of the specified char value.
14	String replace(CharSequence old, CharSequence new)	It replaces all occurrences of the specified CharSequence.
15	static String equalsIgnoreCase(String another)	It compares another string. It doesn't check case.
16	String[] split(String regex)	It returns a split string matching regex.
17	String[] split(String regex, int limit)	It returns a split string matching regex and limit.
18	String intern()	It returns an interned string.
19	int indexOf(int ch)	It returns the specified char value index.
20	int indexOf(int ch, int fromIndex	It returns the specified char value index starting with given index.
21	int indexOf(String substring)	It returns the specified substring index.
22	int indexOf(String substring, int fromIndex)	It returns the specified substring index starting with given index.
23	String toLowerCase()	It returns a string in lowercase.
24	String toLowerCase(Locale l)	It returns a string in lowercase using specified locale.
25	String toUpperCase()	It returns a string in uppercase.
26	String toUpperCase(Locale l)	It returns a string in uppercase using specified locale.
27	String trim()	It removes beginning and ending spaces of this string.
28	static String valueOf(int value)	It converts given type into string. It is an overloaded method.

- The java.lang.String class is used to create string object.
- Java String class provides a lot of methods to perform operations on string such as
  - concat()
  - equals()
  - split()
  - length()
  - replace()
  - compareTo()
  - substring(), etc.

concat(String str):

- The **java string concat()** method *combines specified string at the end of this string*.
- It returns combined string.
- It is like appending another string.

```
public class CombinString
{
  public static void main(String[] args)
  {
     Scanner s=new Scanner(System.in);
    System.out.println("Enter First String:");
    String str1 = s.next();
    System.out.println("Enter Second String:");
    String str2 = s.next();
    System.out.println("Combin string example!");
    String com = str1.concat(str2);
    System.out.println("Combined string: " + com);
  }
}
```

Compare strings example

- The **java lang** package provides a method to compare two with their case either upper and lower.
- The **equals()** method provides the facility of comparing the two strings.
- **equals():**

  This is the method that compares an object values and returns Boolean type value either '**true**' or '**false**'.

  If it returns 'true' for the both objects, it will be equal otherwise not.

```
public class EqualsExample
{
  public static void main(String args[])
  {
        String s1="javatpoint";
        String s2="javatpoint";
        String s3="JAVATPOINT";
        String s4="python";
        System.out.println(s1.equals(s2));
        System.out.println(s1.equals(s3));
        System.out.println(s1.equals(s4));
  }
}
```

String length example

- **length();**
  - This is the method that returns the length of given string in integer type.

```
public class LengthExample
{
   public static void main(String args[])
   {
         String s1="javatpoint";
         String s2="python";
         System.out.println("string length is: "+s1.length());
         System.out.println("string length is: "+s2.length());
   }
}
```

String replace example

- replace();

```
public class ReplaceExample1
{
    public static void main(String args[])
    {
        String s1="javatpoint is a very good website";
        String replaceString=s1.replace('a','e');
        System.out.println(replaceString);
    }
}
```

replace(CharSequence target, CharSequence replacement)

```
public class ReplaceExample2
{
    public static void main(String args[])
    {
        String s1="my name is khan my name is java";
        String replaceString=s1.replace("is","was");
        System.out.println(replaceString);
    }
}
```

Java String equalsIgnoreCase() Method Example

```
public class EqualsIgnoreCaseExample{
public static void main(String args[]){
String s1="javatpoint";
String s2="javatpoint";
String s3="JAVATPOINT";
String s4="python";
System.out.println(s1.equalsIgnoreCase(s2));//true because content and case is same
System.out.println(s1.equalsIgnoreCase(s3));//true because case is ignored
System.out.println(s1.equalsIgnoreCase(s4));//false because content is not same
}}
```

Java String toLowerCase()

```
public class StringLowerExample{
public static void main(String args[]){
String s1="JAVATPOINT HELLO stRIng";
String s1lower=s1.toLowerCase();
System.out.println(s1lower);
}}
```

Java String toUpperCase() method example

```
public class StringUpperExample
{
public static void main(String args[])
{
String s1="hello string";
String s1upper=s1.toUpperCase();
System.out.println(s1upper);
}
}
```

Java String split() method example

```
public class SplitExample
{
public static void main(String args[])
{
String s1="java string split method by javatpoint";
String[] words=s1.split("\\s");
for(String w:words)
{
System.out.println(w);
}
}}
```

Java String substring() method example

```
public class SubstringExample2 {
   public static void main(String[] args) {
      String s1="Javatpoint";
      String substr = s1.substring(0);
      System.out.println(s1.substring(2));
      System.out.println(substr);
      String substr2 = s1.substring(5,10);
      System.out.println(substr2);
      String substr3 = s1.substring(5,15);
   }
}
```

Java String startsWith() method example

```
public class StartsWithExample
{
// main method
public static void main(String args[])
{
// input string
String s1="java string split method by javatpoint";
System.out.println(s1.startsWith("ja"));  // true
System.out.println(s1.startsWith("java string"));   // true
System.out.println(s1.startsWith("Java string"));  // false as 'j' and 'J' ar
e different
}
}
```

Java String trim() Method Example

```
public class StringTrimExample
{
public static void main(String args[])
{
String s1="  hello string   ";
System.out.println(s1+"javatpoint");//without trim()
System.out.println(s1.trim()+"javatpoint");//with trim()
}
}
```

```
hello string javatpoint
hello stringjavatpoint
```

Java String indexOf(String substring) Method Example

```
public class IndexOfExample2 {
	public static void main(String[] args) {
		String s1 = "This is indexOf method";

		// Passing Substring
		int index = s1.indexOf("indexOf"); //Returns the
index of this substring
		System.out.println("index of substring "+index);

	}

}
```

index of substring 8

StringBuffer

StringBuffer

- The StringBuffer class is used to represent characters that can be modified.
- This is simply used for concatenation or manipulation of the strings.
- StringBuffer is mainly used for the dynamic string concatenation which enhances the performance.
- A string buffer implements a mutable sequence of characters.
- A string buffer is like a String but can be modified.

No	String	StringBuffer
1)	String class is immutable.	StringBuffer class is mutable.
2)	String is slow and consumes more memory when you concat too many strings because every time it creates new instance.	StringBuffer is fast and consumes less memory when you cancat strings.
3)	String class overrides the equals() method of Object class. So you can compare the contents of two strings by equals() method.	StringBuffer class doesn't override the equals() method of Object class.

StringBuffer Functions/Methods

- **append()**
  - This is the append() function used for the concatenate the string in string buffer.
  - This is better to use for dynamic string concatenation.
  - This function works like a simple string concatenation such as : String str = str + "added string";.

insert()

- This function is used to insert any string or character at the specified position in the given string.

reverse()

- This function is used to reverse the string present in string buffer.

setCharAt()

- This function is used to set the specified character in buffered string at the specified position of the string in which you have to set the given character.

charAt()

- This is the charAt() function which is used to get the character at the specified position of the given string.

substring()

- This function is used to get the sub string from the buffered string from the initial position to end position.

deleteCharAt()

- This function is used to delete the specific character from the buffered string by mentioning that's position in the string.

length()

- This function is used to find the length of the buffered string.

delete()

- This function is used to delete multiple character at once from n position to m position (n and m are will be fixed by you.) in the buffered string.

capacity()

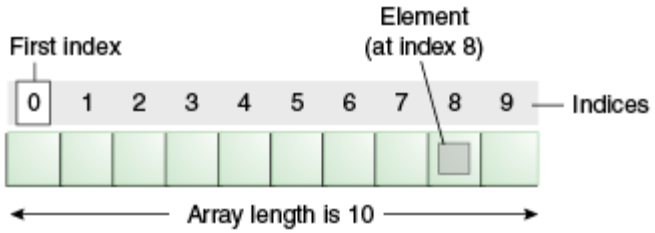
- This function is used to know about the current characters kept which is displayed like : number of characters + 6.

No.	Hands-on Assignment	Topics Covered
1	Write a Program to check whether a given String is Palindrome or not.	String/StringBuffer
2	Write a java program that will concatenate 2 strings and return the result. The result should be in lowercase. Note:If the concatenation creates a double-char, then one of the characters need to be omitted. Example1) i/p:Sachin,Tendulkar o/p:sachin tendulkar Example2) i/p:Mark,kate o/p:markate	String/StringBuffer
3	Given a string, return a new string made of 'n' copies of the first 2 chars of the original string where 'n' is the length of the string. Example1) i/p:Wipro o/p:WiWiWiWiWi	String/StringBuffer
4	Write a java program that will return the first half of the string, if the length of the string is even. It should return null for odd length string. Example1) i/p:TomCat o/p:Tom Example2) i/p:Apron o/p:null	String/StringBuffer

5	Write a java program that accepts a string and returns a new string without the first and last character of the input string. Example1) i/p:Suman o/p:uma	String/StringBuffer
6	Given 2 strings, a and b, return a new string of the form short+long+short, with the shorter string on the outside and the longer string on the inside. The strings will not be the same length, but they may be empty (length 0). If input is "hi" and "hello", then output will be "hihellohi".	String/StringBuffer
7	Given a string, if the first or last chars are 'x', return the string without those 'x' chars, otherwise return the string unchanged. If the input is "xHix", then output is "Hi". If the input is "America", then the output is "America".	String/StringBuffer
8	Write a Java program that accepts a string (with * in it). The program should return a new string in which the following characters are removed-*,the characters that are to the left and right of * Example1) i/p:ab*cd o/p:ad	String/StringBuffer
9	Given two strings, a and b, print a new string which is made of the following combination-first character of a, the first character of b, second character of a, second character of b and so on. Any characters left, will go to the end of the result. Example1) i/p:Hello World	String/StringBuffer

Array in Java

- Array is an object which contains elements of a similar data type.
- We can store only a fixed set of elements in a Java array.



Two Types of Array

- Single Dimensional Array
- Multidimensional Array

Single Dimension Array

Syntax to Declare an Array in Java

dataType[] arr; (or)

dataType arr[];

Instantiation of an Array in Java

arrayRefVar=**new** datatype[size];

```
class Testarray
{
public static void main(String args[])
{
        int a[]=new int[5];
        a[0]=10;//initialization
        a[1]=20;
        a[2]=70;
        a[3]=40;
        a[4]=50;
for(int i=0;i<a.length;i++)
                System.out.println(a[i]);
}
}
```

```
java -cp /tmp/D4vWZpN0BI
Testarray

10
20
70
40
50
```

For-Each Array

```
class Testarray1
{
public static void main(String args[])
{
        int arr[]={33,3,4,5};
        //printing array using for-each loop
        for(int i:arr)
        System.out.println(i);
        }
}
```

Passing Array to a Method in Java

```
class Testarray2
{
//creating a method which receives an array as a parameter
static void min(int arr[])
{
        int min=arr[0];
        for(int i=1;i<arr.length;i++)
         if(min>arr[i])
          min=arr[i];
        System.out.println(min);
}

public static void main(String args[])
{
int a[]={33,3,4,5};          //declaring and initializing an array
min(a);                       //passing array to method
}
}
```

Output: 3

Returning Array from the Method

```
class TestReturnArray
{
//creating method which returns an array
static int[] get()
{
    return new int[]{10,30,50,90,60};
}

public static void main(String args[])
{
//calling method which returns an array
        int arr[]=get();
//printing the values of an array
        for(int i=0;i<arr.length;i++)
        System.out.println(arr[i]);
}
}
```

Multidimensional Array in Java

Data is stored in row and column based index (also known as matrix form).

Syntax:

```
dataType[][] arrayRefVar; (or)
dataType [][]arrayRefVar; (or)
dataType arrayRefVar[][]; (or)
dataType []arrayRefVar[];
```

Example to instantiate Multidimensional Array in Java

```
int[][] arr=new int[3][3];
```

Example to initialize Multidimensional Array in Java

```
arr[0][0]=1;
arr[0][1]=2;
arr[0][2]=3;
arr[1][0]=4;
arr[1][1]=5;
arr[1][2]=6;
arr[2][0]=7;
arr[2][1]=8;
arr[2][2]=9;
```

1 2 3 2 4 5 4 4 5

```
class Testarray3
{
public static void main(String args[])
{
//declaring and initializing 2D array
    int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
//printing 2D array
    for(int i=0;i<3;i++){
    for(int j=0;j<3;j++){
  System.out.print(arr[i][j]+" ");
 }
 System.out.println();
}
}
}
```

Compile by: javac Testarray3.java
Run by: java Testarray3

```
1 2 3
2 4 5
4 4 5
```

Multiplication of 2 Matrices in Java

Matrix 1

$$\begin{Bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{Bmatrix}$$

Matrix 2

$$\begin{Bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{Bmatrix}$$

Matrix 1 * Matrix 2

$$\begin{Bmatrix} 1*1+1*2+1*3 & 1*1+1*2+1*3 & 1*1+1*2+1*3 \\ 2*1+2*2+2*3 & 2*1+2*2+2*3 & 2*1+2*2+2*3 \\ 3*1+3*2+3*3 & 3*1+3*2+3*3 & 3*1+3*2+3*3 \end{Bmatrix}$$

Matrix 1 * Matrix 2

$$\begin{Bmatrix} 6 & 6 & 6 \\ 12 & 12 & 12 \\ 18 & 18 & 18 \end{Bmatrix}$$

JavaTpoint

```java
class TestJaggedArray
{
   public static void main(String[] args)
{
      //declaring a 2D array with odd columns
     int arr[][] = new int[3][];
      arr[0] = new int[3];
      arr[1] = new int[4];
      arr[2] = new int[2];
      //initializing a jagged array
      int count = 0;
      for (int i=0; i<arr.length; i++)
         for(int j=0; j<arr[i].length; j++)
            arr[i][j] = count++;
        //printing the data of a jagged array
      for (int i=0; i<arr.length; i++){
         for (int j=0; j<arr[i].length; j++){
            System.out.print(arr[i][j]+" ");
         }
         System.out.println();//new line
      }
   }
}
```

Compile by: javac TestJaggedArray.java
Run by: java TestJaggedArray

```
0 1 2
3 4 5 6
7 8
```

```
class Testarray5
{
    public static void main(String args[])
      {
      //creating two matrices
      int a[][]={{1,3,4},{3,4,5}};
      int b[][]={{1,3,4},{3,4,5}};

      //creating another matrix to store the sum of two matrices
      int c[][]=new int[2][3];

      //adding and printing addition of 2 matrices
            for(int i=0;i<2;i++)
            {
                        for(int j=0;j<3;j++){
                        c[i][j]=a[i][j]+b[i][j];
                        System.out.print(c[i][j]+" ");
            }
            System.out.println();//new line
            }
}
}
```

Compile by: javac Testarray5.java
Run by: java Testarray5
2 6 8
6 8 10

Inner Class

Inner Class

- The purpose of nested classes is to group classes that belong together, which makes your code more readable and maintainable.
- To access the inner class, create an object of the outer class, and then create an object of the inner class:

```
class OuterClass {
  int x = 10;

  class InnerClass {
    int y = 5;
  }
}

public class Main {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.y + myOuter.x);
  }
}
```

Static Inner Class

```
class OuterClass {
  int x = 10;

  static class InnerClass {
    int y = 5;
  }
}

public class Main {
  public static void main(String[] args) {
    OuterClass.InnerClass myInner = new OuterClass.InnerClass();
    System.out.println(myInner.y);
  }
}
```

Access Outer Class From Inner Class

```
class OuterClass {
  int x = 10;

  class InnerClass {
    public int myInnerMethod() {
      return x;
    }
  }
}

public class Main {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.myInnerMethod());
  }
}
```

1	Write a program to initialize an integer array and print the sum and average of the array.
2	Write a program to initialize an integer array and find the maximum and minimum value of the array.
3	Write a program to initialize an integer array with values and check if a given number is present in the array or not. If the number is not found, it will print -1 else it will print the index value of the given number in the array. Example 1) If the Array elements are {1,4,34,56,7} and the search element is 90, then the output expected is -1. Example 2)If the Array elements are {1,4,34,56,7} and the search element is 56, then the output expected is 3.
4	Initialize an integer array with ascii values and print the corresponding character values in a single row.
5	Write a program to find the largest 2 numbers and the smallest 2 numbers in the given array.
6	Write a program to initialize an array and print them in a sorted order.
7	Write a program to remove the duplicate elements in an array and print the same. Example) I/P:{12,34,12,45,67,89} O/P:{12,34,45,67,89}

8	If the array has 6 and 7 in succeeding orders, ignore the numbers between 6 and 7 and consider the other numbers for calculation of sum. Eg1) Array Elements - 10,3,6,1,2,7,9 O/P: 22 [i.e 10+3+9] Eg2) Array Elements - 7,1,2,3,6 O/P:19 Eg3) Array Elements - 1,6,4,7,9 O/P:10	One dimensional Array
9	Print a version of the given array where all the 10's have been removed. The remaining elements should shift left towards the start of the array as needed, and the empty spaces at the end of the array should be 0. So {1, 10, 10, 2} yields {1, 2, 0, 0}. You may modify and display the given array or make a new array. withoutTen([1, 10, 10, 2]) → [1, 2, 0, 0] withoutTen([10, 2, 10]) → [2, 0, 0] withoutTen([1, 99, 10]) → [1, 99, 0]	One dimensional Array
10	Print an array that contains the exact same numbers as the given array, but rearranged so that all the even numbers come before all the odd numbers. Other than that, the numbers can be in any order. You may modify and print the given array, or make a new array. evenOdd([1, 0, 1, 0, 0, 1, 1]) → [0, 0, 0, 1, 1, 1, 1] evenOdd([3, 3, 2]) → [2, 3, 3] evenOdd([2, 2, 2]) → [2, 2, 2]	One dimensional Array
11	Given an array of type int, print true if every element is 1 or 4. only14([1, 4, 1, 4]) → true only14([1, 4, 2, 4]) → false only14([1, 1]) → true	One dimensional Array

12	Given 2 int arrays, a and b, each length 3, form a new array of length 2, containing their middle elements. middleWay([1, 2, 3], [4, 5, 6]) → [2, 5] middleWay([7, 7, 7], [3, 8, 0]) → [7, 8] middleWay([5, 2, 9], [1, 4, 5]) → [2, 4]	One dimensional Array	(
13	Write a program to reverse the elements of a given 2*2 array. Four integer numbers needs to be passed as Command Line arguments. Example1) C:\>java Sample 1 2 3 O/P: Please enter 4 integer numbers Example2) C:\>java Sample 1 2 3 4 O/P: The given array is : 1 2 3 4 The reverse of the array is : 4 3 2 1	Two Dimensional Array	(
14	Write a program to find the biggest number in a 3*3 array. The program is supposed to receive 9 integer numbers as command line arguments. Example1: C:\>java Sample 1 2 3 O/P: Please enter 9 integer numbers Example2: C:\>java Sample 1 23 45 55 121 222 56 77 89 O/P: The given array is : 1 23 45 55 121 222 56 77 89 The biggest number in the given array is 222	Two Dimensional Array	(

Stream : JAVA **Tech Module : 1** **Project:1**

Given the following table containing information about employees of an organization, develop a small java application, which accepts employee id from the command prompt and displays the following details as output:

EmpNo EmpName Department Designation Salary

In your program, you must initialize an array with the following details.

Emp No	Emp Name	Join Date	Designation Code	Department	Basic	HRA	IT
1001	Ashish	01/04/2009	e	R&D	20000	8000	3000
1002	Sushma	23/08/2012	c	PM	30000	12000	9000
1003	Rahul	12/11/2008	k	Acct	10000	8000	1000
1004	Chahat	29/01/2013	r	Front Desk	12000	6000	2000
1005	Ranjan	16/07/2005	m	Engg	50000	20000	20000
1006	Suman	1/1/2000	e	Manufacturing	23000	9000	4400
1007	Tanmay	12/06/2006	c	PM	29000	12000	10000

DA (Dearness Allowance) details are given in the below "Designation" table.

Designation Code	Designation	DA
e	Engineer	20000
c	Consultant	32000
k	Clerk	12000
r	Receptionist	15000
m	Manager	40000

DA (Dearness Allowance) details are given in the below "Designation" table.

Designation Code	Designation	DA
e	Engineer	20000
c	Consultant	32000
k	Clerk	12000
r	Receptionist	15000
m	Manager	40000

Note 1: Salary should be calculated as (Basic + HRA + DA – IT).
Note 2: Use switch-case to print Designation and to find the value of DA for a particular employee.

Expected Output format: (assuming that your class name is Project1)

- If you execute the command **java Project1 1003**, the output should be –

```
Emp No.  Emp Name   Department   Designation   Salary
1003     Rahul      Acct         Clerk         29000
```

- If you execute the command **java Project1 123**, the output should be –

```
There is no employee with empid : 123
```